Algebraic Server Reconciliation for Online Games

Algebraische serverseitige Zustandskorrektur für Online-Spiele
Master thesis by Philipp Hinz
Date of submission: September 29, 2025

1. Review: Dr.-Ing. Guido Rößling
Darmstadt

TECHNISCHE
UNIVERSITÄT
DARMSTADT

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Philipp Hinz, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§38 Abs.2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 29.09.2025

Philipp Hinz

Abstract

Real-time networked multiplayer games must maintain a consistent game state across clients despite network latency. The predominant solution, client-side prediction with rollback reconciliation, provides responsiveness by speculatively simulating player actions and then correcting mispredictions by re-simulating from authoritative server states. This approach, while effective, imposes significant computational overhead that scales linearly (O(n)) with the number of frames requiring re-simulation.

This thesis introduces algebraic server reconciliation, a novel method that eliminates this re-simulation overhead. By representing state changes as elements of an abelian group, our method enables the direct algebraic correction of predicted states through a single mathematical operation, reducing computational complexity to constant time (O(1)) per reconciliation event. We provide a formal framework for constructing suitable abelian groups for Entity Component System (ECS) architectures and prove the method's convergence.

Experimental evaluation demonstrates that algebraic reconciliation achieves synchronization quality comparable to rollback for position-based movement. While the method exhibits characteristic overcorrection artifacts during complex physics interactions that violate its underlying assumptions, performance analysis confirms the elimination of re-simulation overhead. Our work adapts concepts from Conflict-Free Replicated Data Types (CRDTs) to client-server game architectures, offering a computationally efficient alternative for genres such as MOBAs, RTS games, and large-scale battle royales where its mathematical assumptions align with gameplay mechanics.

Contents

1 Introduction **1**

2 Related Work **3**
2.1 Foundational Architectures for Latency Mitigation . . . 3
2.1.1 The Authoritative Server and the Challenge of Latency . . . 3
2.1.2 Client-Side Prediction and Server Reconciliation . . . 4
2.1.3 Rollback Netcode . . . 5
2.1.4 Server-Side Lag Compensation . . . 6
2.2 Deterministic Synchronization Models . . . 7
2.2.1 The Deterministic Lockstep Architecture . . . 7
2.2.2 Analysis of Trade-Offs and Application . . . 8
2.3 Theoretical Frameworks for State Convergence . . . 9
2.3.1 Strong Eventual Consistency in Peer-to-Peer Gaming . . . 9
2.3.2 Conflict-Free Replicated Data Types (CRDTs) . . . 9
2.4 Case Studies of Networking in Practice . . . 10
2.4.1 Valve's Source Engine: A Canonical Implementation . . . 10
2.4.2 Overwatch: Determinism in a High-Paced Shooter . . . 11
2.5 Summary and Research Opportunities . . . 12

3 Network model **13**
3.1 Overview . . . 13
3.1.1 Naive model . . . 15
3.1.2 Fair naive model . . . 16
3.1.3 Lockstep . . . 17
3.1.4 Player observation limitation . . . 18
3.1.5 Client side prediction . . . 19
3.1.6 Server reconciliation . . . 21
3.1.7 Delta State . . . 24
3.2 Algebraic server reconciliation . . . 26
3.2.1 Overview . . . 26
3.2.2 Definition . . . 27
3.2.3 Limitations . . . 29
3.2.4 Abelian group for ECS games . . . 30

4 Networking Architecture **33**
4.1 Overview . . . 33
4.1.1 Four quadrants model . . . 34
4.1.2 Server and Client . . . 35
4.1.3 Game and Network . . . 35
4.2 Server networking layer . . . 36
4.3 Client networking layer . . . 38

4.3.1 Rollback reconciliation . . . 40
4.3.2 Algebraic reconciliation . . . 41

5 Experiments **43**
5.1 Methodology . . . 43
5.2 Experiment 1: Isolated Movement Patterns . . . 44
5.2.1 Setup . . . 44
5.2.2 Results . . . 44
5.2.3 Analysis . . . 46
5.3 Experiment 2: Collision Dynamics and State Dependencies . . . 47
5.3.1 Setup . . . 47
5.3.2 Results . . . 48
5.3.3 Analysis . . . 52
5.4 Discussion . . . 53
5.4.1 The Landscape of Networking Optimizations . . . 53
5.4.2 Trade-offs in Network Architecture Design . . . 53
5.4.3 Connections to Established Distributed Systems Techniques . . . 54
5.4.4 Limitations and Domain Constraints . . . 54
5.4.5 Applicability and Adoption Potential . . . 55
5.4.6 Toward Hybrid Network Architectures . . . 55
5.4.7 Implications for Game Development Practice . . . 56

6 Conclusion and Future work **57**
6.1 Contributions and Broader Impact . . . 57
6.2 Hybrid Reconciliation Architectures . . . 58
6.3 Industrial Validation Through Full Game Implementation . . . 58
6.4 Merkle Tree Optimization for Selective State Transmission . . . 59
6.5 Higher-Order Derivative Synchronization . . . 59
6.6 Control Theory Applications for Correction Stability . . . 60
6.7 Predictive Network Modeling . . . 60
6.8 Formal Verification and Property-Based Testing . . . 60
6.9 Conclusion . . . 61

1 Introduction

The rise of online multiplayer gaming has transformed interactive entertainment, connecting millions of players in shared virtual worlds. This transformation, however, introduces a fundamental technical challenge: the physical reality of network latency. When a player in Berlin battles an opponent in Tokyo, their actions must traverse thousands of kilometers of network infrastructure, introducing delays that can render fast-paced games unplayable if not properly mitigated.

The challenge is not merely technical but experiential. Modern players expect instantaneous response to their inputs, yet in a networked environment, this expectation conflicts with the need for consistency across all players. This tension has driven the development of sophisticated synchronization techniques to hide or compensate for the irreducible reality of network latency.

The current industry standard, client-side prediction with rollback reconciliation, represents an elegant but computationally expensive solution. Clients optimistically predict the results of their actions for immediate visual feedback. When the server's response reveals a misprediction, inevitable when clients lack complete information, the client "rolls back" its state and re-simulates all intervening frames with the corrected information. This approach successfully maintains both responsiveness and eventual consistency, but at a cost: the computational burden of repeated simulation increases linearly with network latency. This overhead is particularly problematic for mobile devices with limited battery and thermal budgets, and for cloud gaming platforms operating at a massive scale.

This thesis presents algebraic server reconciliation, a novel approach to state synchronization that maintains the responsiveness of client-side prediction while eliminating the computational overhead of re-simulation. Our key insight is that by representing game state changes as elements of an abelian group, a mathematical structure with well-defined properties of composition and inversion, we can compute corrections directly through algebraic operations rather than repeated simulation. When a client receives an authoritative server update, instead of rolling back and re-simulating, it computes a single correction delta that transforms its current predicted state toward convergence with the server.

Our approach draws inspiration from Conflict-Free Replicated Data Types (CRDTs), a technique from distributed systems that leverages algebraic properties to ensure eventual consistency. However, unlike CRDTs which typically handle peer-to-peer updates, we exploit the authoritative server topology common in games to relax requirements and support a broader class of operations.

The primary contributions of this thesis are threefold. First, we introduce the algebraic server reconciliation method with formal proofs of convergence and complexity analysis. Second, we provide a comprehensive formalization of game networking concepts that enables rigorous comparison between different synchronization approaches. Third, we present experimental validation using custom-built test games that reveals both the strengths and limitations of our method under various gameplay scenarios.

Through this investigation, we demonstrate that algebraic server reconciliation offers a viable alternative to traditional rollback for a significant class of multiplayer games. While not universally applicable - certain game mechanics violate its underlying assumptions - the technique provides valuable benefits where its requirements align with gameplay design. By expanding the palette of synchronization techniques, this work contributes to the ongoing evolution of networked interactive entertainment.

2 Related Work

The challenge of maintaining a consistent and responsive shared reality across a distributed system is a foundational problem in computer science. In the domain of real-time networked multiplayer games, this challenge is amplified by stringent latency requirements and the demand for interactive fairness. Fast-paced shooters are a quintessential example, where success is often decided in milliseconds and perceived fairness is paramount.

This chapter reviews practical state-synchronization techniques for real-time games alongside relevant distributed-systems results. Academic work covers theory (consistency, replication, CRDTs), while many production details are documented in engine manuals, GDC talks [4], and developer blogs. We therefore integrate peer-reviewed sources with primary industry material, citing each claim to a concrete talk or manual where possible.

The chapter begins by examining the foundational architectures for latency mitigation in the prevalent client-server model. It then contrasts this with the alternative paradigm of deterministic synchronization. Subsequently, it explores theoretical frameworks from distributed systems research, such as Conflict-Free Replicated Data Types (CRDTs) [9], that offer alternative models for state convergence. Finally, it grounds these discussions in case studies of influential, real-world implementations.

2.1 Foundational Architectures for Latency Mitigation

To mitigate perceived latency, modern games hide delay rather than remove it. We focus on three core strategies: client-side prediction (local responsiveness), reconciliation (correcting divergence), and server-side lag compensation (fair hit evaluation).

2.1.1 The Authoritative Server and the Challenge of Latency

The predominant architecture for modern multiplayer games is the client-server model, wherein a single server is designated as the authoritative source of truth for the game state. Clients send user inputs to the server and, in return, receive periodic updates, or “snapshots”, of the world state. This model centralizes simulation and prevents many forms of cheating, as the client is never fully trusted with game-critical logic. However, it introduces the fundamental problem of network latency: the round-trip time (RTT), illustrated in Figure 2.1, which is the total time it takes for a client’s input to reach the server and for the server’s response to return. For fast-paced action games, where delays of even a few milliseconds can be perceptible, this latency renders a naive implementation unplayable, as there would be a significant delay between a player’s action and its visual feedback. The following sections detail the primary techniques developed to conceal or compensate for this inherent latency.

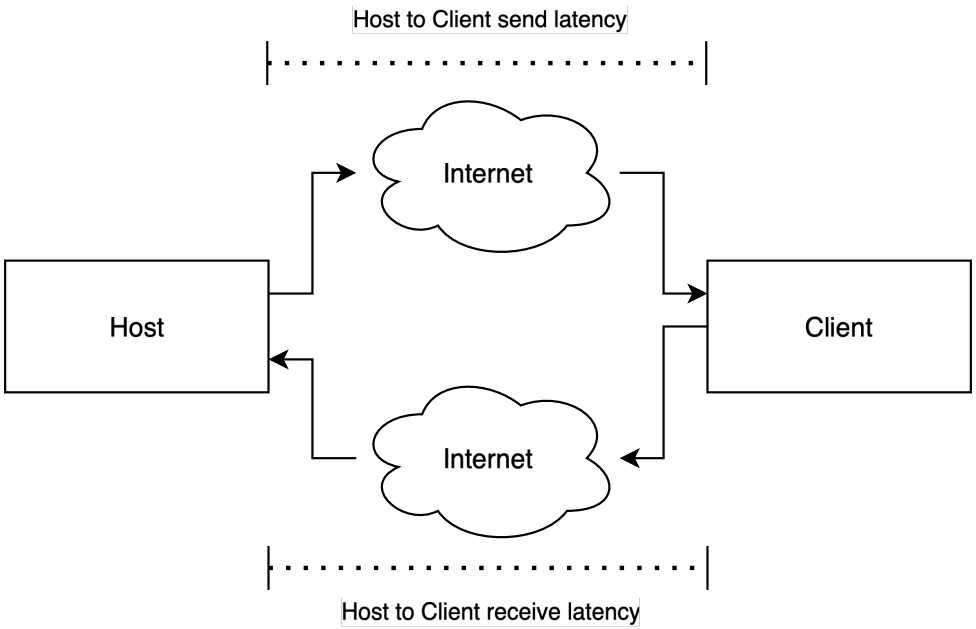

Figure 2.1: Illustration of network latency between a host and a client. The round-trip time (RTT) is the sum of the send and receive latencies.

2.1.2 Client-Side Prediction and Server Reconciliation

To combat the feeling of unresponsiveness, the most widely adopted technique is Client-Side Prediction. The client does not wait for the server's confirmation to execute a local player's command. Instead, it speculatively executes the input and simulates the outcome locally, providing immediate visual feedback. This creates the illusion of a zero-latency environment for the local player's movement and actions. The earliest known implementation in a first-person shooter was in Duke Nukem 3D (1996), as shown in Figure 2.2, with the technique being popularized by id Software's QuakeWorld [7].

Figure 2.2: A screenshot from *Duke Nukem 3D* (1996). The game is one of the earliest known first-person shooters to implement client-side prediction, providing players with immediate visual feedback for their movement to hide network latency.

This speculative execution, however, introduces the problem of divergence. The client's predicted state may differ from the server's authoritative state, which has perfect information about the world, including the actions of other players and collision physics. This discrepancy is known as a prediction error. To resolve this, a process called Server Reconciliation is employed. When the client receives an authoritative state update from the server, it compares this state with its own predicted state. If a mismatch is detected, the client must correct its state to match the server's. A naive correction would result in a jarring visual "snap" as the player character is teleported to the correct position. To mitigate this, a crucial step is performed: the client replays all the inputs that were sent to the server but have not yet been acknowledged in the received server snapshot. By reapplying these inputs from the corrected server state, the client can re-simulate its way back to the present, often resulting in a correct final position and eliminating the visual artifact entirely in most cases. This process requires clients to send inputs with a sequence number and for the server to echo back the number of the last processed input.

2.1.3 Rollback Netcode

Rollback netcode can be understood as a more aggressive and generalized application of the principles of prediction and reconciliation, primarily developed to meet the stringent timing requirements of fighting games. While traditional client-side prediction is typically applied only to the local player, rollback netcode predicts the inputs of the remote player as well, often by assuming they will repeat their last known input. The game simulation for all players advances based on these local and predicted inputs. When an actual input from the remote player arrives, the game state is 'rolled back' to the frame before the prediction began, the correct input is inserted, and the simulation is 'fast-forwarded' back to the present time. This multi-step correction process is visualized in Figure 2.3. The entire rollback and re-simulation ideally occurs within a single frame, making the correction imperceptible to the user.

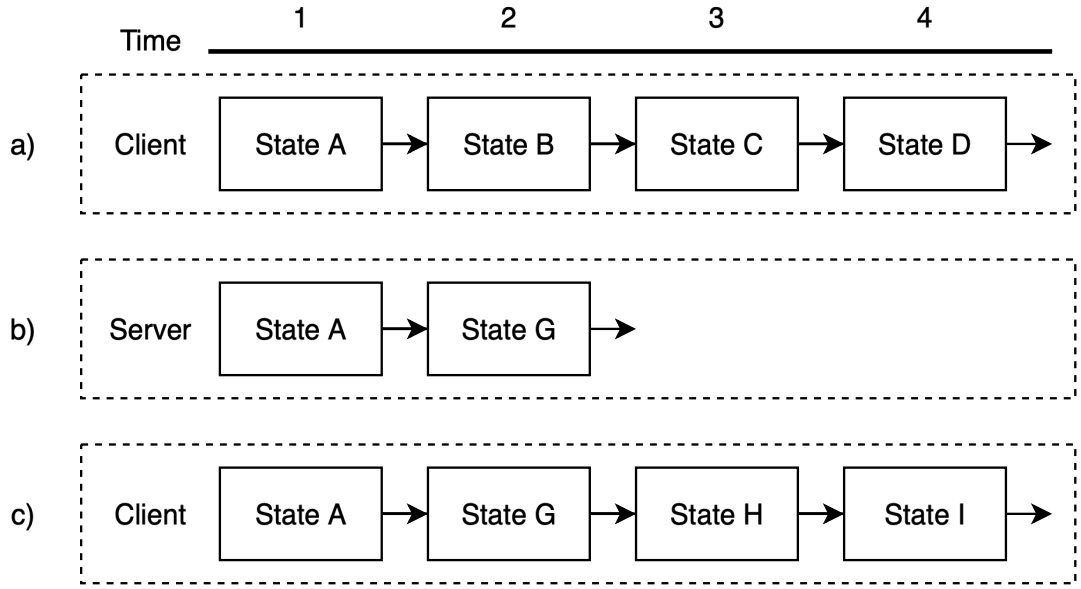

Figure 2.3: Illustration of the rollback netcode process. (a) The client's initial predicted state progression. (b) The server's authoritative state diverges (State G instead of B). (c) Upon receiving the authoritative State G, the client rolls back to State A, applies State G, and then re-simulates its subsequent inputs to reach States H and I.

The primary benefit is a highly responsive experience that feels akin to offline play, even under significant latency. However, the implementation is complex, requiring the ability to save and load the complete game state very rapidly for every frame and to re-simulate multiple frames' worth of logic in a fraction of a second. While rollbacks aim to be imperceptible, extreme cases or visual debuggers can reveal the momentary "snapping" as shown in Figure 2.4. Michael Stallone of NetherRealm Studios noted that retrofitting rollback into Mortal Kombat X took approximately two man-years of effort, with significant challenges in optimizing serialization and ensuring non-deterministic elements like visual and audio effects were handled consistently during rollbacks [10].

Figure 2.4: A screenshot from the fighting game *Mortal Kombat X* (2015). The genre's demand for frame-perfect inputs makes it extremely sensitive to latency, establishing fighting games as a primary driver for the adoption of rollback netcode. The challenges of retrofitting this technology into *Mortal Kombat X* are a well-documented example of its implementation complexity.

2.1.4 Server-Side Lag Compensation

While client-side prediction addresses the local player's sense of responsiveness, it does not solve the problem of interactional fairness. A player might fire at a target that is clearly visible on their screen, but due to latency, the target has already moved to a new position on the server's timeline by the time the shot command is processed. This fundamental time discrepancy is illustrated in Figure 2.5.

To solve this, servers employ Lag Compensation. The server maintains a short history of past player positions. When it receives a user command, such as a shot, it uses the packet's latency to estimate the time at which the command was actually executed on the client. It then temporarily "rewinds" the positions of other players in the world to where they were at that moment in the past. The hit detection is then performed against these historical positions. After the command is processed, the players are returned to their current positions.

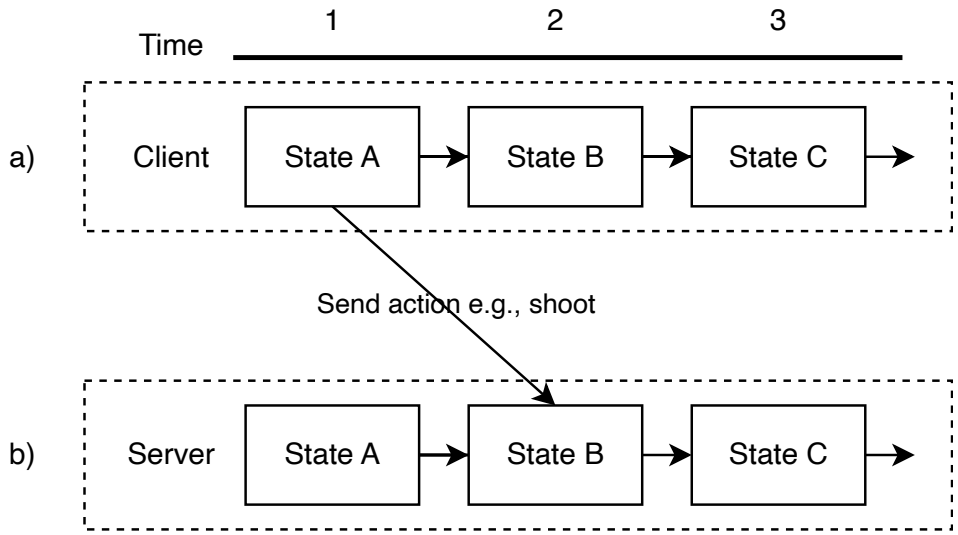

Figure 2.5: Visualizing the time discrepancy that necessitates server-side lag compensation. (a) The client performs an action, such as shooting, at Time 1 based on their local view of the game. (b) Due to network latency, the server receives this action at a later point, Time 2. Lag compensation allows the server to "rewind" the game state to evaluate the client's action as it occurred in the past, ensuring the outcome matches what the player saw.

This technique, famously implemented in the Source Engine, is often described as "favor the shooter" [11]. It ensures that if a player sees a valid target, their shot will register, creating a more reliable and fair experience for the attacker. However, it can lead to perceptual paradoxes for the victim, who may be hit after they have already taken cover on their own screen. This is an accepted trade-off in most modern shooters, prioritizing interactional consistency over strict temporal accuracy.

The collection of these techniques reveals a core design duality in networked games. One branch of techniques, encompassing prediction, reconciliation, and rollback, is focused on optimizing the local player's subjective feeling of control and responsiveness. The other branch, exemplified by lag compensation, is focused on optimizing the acting player's objective sense of fairness during interactions. These two goals can be in direct conflict, and the specific balance chosen by a game's networking model defines much of its "feel".

2.2 Deterministic Synchronization Models

In sharp contrast to sending frequent state updates, deterministic models synchronize only player inputs. This section explores architectures where each client runs an identical simulation. Provided each client processes the same inputs in the same order, their worlds remain perfectly synchronized. We will examine the classic lockstep model, its profound bandwidth advantages, and its demanding trade-offs, such as the strict requirement for perfect determinism.

2.2.1 The Deterministic Lockstep Architecture

In contrast to the state-synchronization model employed by authoritative servers, the deterministic lockstep model operates on a fundamentally different principle. Instead of transmitting game state, clients only transmit their inputs to one another, either through a peer-to-peer topology or

relayed via a simple server. Each client runs an identical, fully deterministic simulation of the game world. Provided that each client starts from the same initial state and processes the exact same sequence of inputs in the exact same order (at the same "tick"), their game states are guaranteed to remain perfectly synchronized.

To ensure inputs are processed in the same order, the simulation proceeds in discrete turns. The game will not advance to the next turn until it has received the inputs from all players for that turn. As depicted in Figure 2.6, this means the entire simulation is forced to pause and wait for the most delayed participant. To hide the latency of waiting for these inputs, games implementing this model often introduce a fixed input delay, buffering inputs for a short period before executing them. This model's primary advantage is its extraordinary bandwidth efficiency concerning the number of game entities. Since only small input packets are transmitted, the network load scales with the number of players, not the number of units on screen, making it ideal for genres with potentially thousands of entities, such as Real-Time Strategy (RTS) games [1].

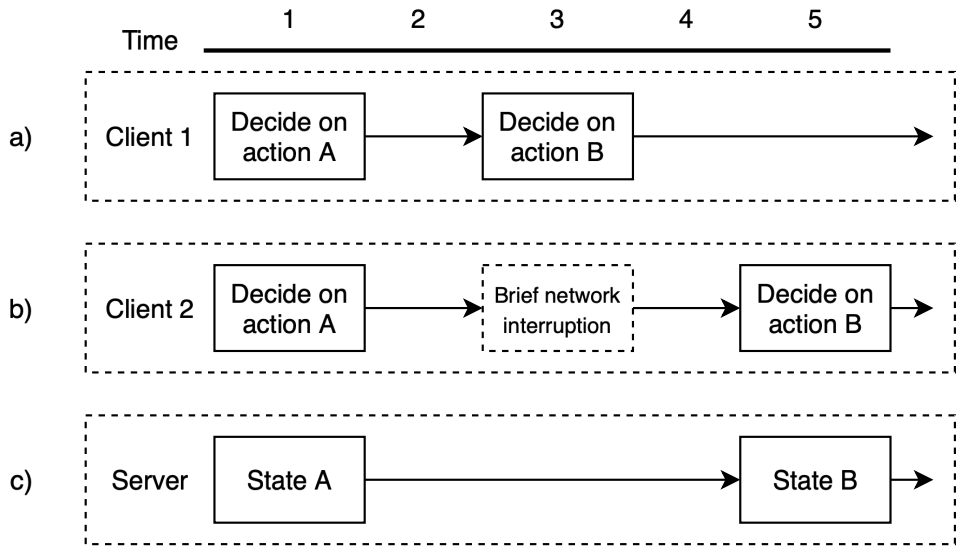

Figure 2.6: Illustration of the lockstep model and its sensitivity to network conditions. (a) Client 1 processes its actions seamlessly. (b) Client 2 experiences a brief network interruption between Time 1 and Time 2, delaying its ability to provide input. (c) The server (or host) cannot advance its state (from State A to State B) until inputs from both Client 1 and Client 2 for the current frame are received. This forces the entire simulation to pause and wait for the slowest or most delayed participant.

2.2.2 Analysis of Trade-Offs and Application

The benefits of the lockstep model come with severe trade-offs. The entire system is only as fast as the slowest participant; if one player's input packet is delayed, the simulation for all players must freeze until it arrives. This makes the model highly susceptible to players with high latency or unstable connections and limits its practical application to games with a relatively low player count.

Furthermore, the requirement of perfect determinism is a formidable software engineering challenge. The simulation must produce bit-identical results across different hardware, operating systems, and compiler settings. This precludes the use of common non-deterministic elements, most notably standard floating-point arithmetic, which can yield slightly different results on

different processor architectures. Developers must rely on fixed-point math and carefully control sources of randomness and data structure ordering to prevent desynchronization, or "desyncs," which are catastrophic failures in this model. A modern evolution, deterministic rollback, combines this model with the rollback techniques described earlier, predicting inputs to avoid freezing and resimulating when the actual inputs arrive [3].

The choice between an authoritative server model and a deterministic lockstep model represents a fundamental architectural decision to shift the primary burden of complexity. The authoritative server model places complexity in the network layer: managing high bandwidth, compressing state, interpolating between snapshots, and reconciling prediction errors. The deterministic lockstep model places complexity in the application layer: enforcing perfect determinism, managing fixed-point math, and dealing with the game design implications of input delay and simulation freezes. The state-sync model effectively says, "Let the application be complex and non-deterministic; we will solve the resulting inconsistencies at the network level with authority and correction." The input-sync model says, "Let's make the network problem trivial by enforcing extreme constraints on the application itself." The choice depends entirely on the game's genre and design priorities.

2.3 Theoretical Frameworks for State Convergence

Beyond bespoke gaming solutions, the field of distributed systems provides formal frameworks for state synchronization. This section introduces these theoretical concepts, focusing on Strong Eventual Consistency (SEC) and a powerful data structure that enables it: the Conflict-Free Replicated Data Type (CRDT). We will explore how CRDTs mathematically guarantee that disparate clients will eventually reach the same state, offering a robust model for decentralized or peer-to-peer architectures.

2.3.1 Strong Eventual Consistency in Peer-to-Peer Gaming

Beyond the specific implementations in gaming, the field of distributed systems provides a theoretical foundation for reasoning about state convergence. A key concept is Strong Eventual Consistency (SEC), which guarantees that if a set of replicas has received the same set of updates, they will be in the same state. Unlike strong consistency models which may require coordination or locking before an operation can be performed, eventual consistency allows replicas to be updated independently and asynchronously, with convergence guaranteed over time. This model is highly relevant for decentralized or peer-to-peer (P2P) game architectures, which lack a central authoritative server to resolve conflicts. The challenge lies in designing data structures and operations that can guarantee this convergence property automatically.

2.3.2 Conflict-Free Replicated Data Types (CRDTs)

Conflict-Free Replicated Data Types (CRDTs) are a class of data structures designed to provide SEC [9]. They allow for concurrent updates on different replicas without coordination, and provide a mathematically proven guarantee that the replicas will converge. This is achieved by designing operations that possess specific algebraic properties.

There are two primary forms of CRDTs: State-based CRDTs (Convergent Replicated Data Types, or CvRDTs): Each replica can send its entire state to another replica. The receiving replica merges

the incoming state with its own using a merge function. To guarantee convergence, this merge function must be associative, commutative, and idempotent. A set of states and a merge function with these properties form a mathematical structure known as a join-semilattice.

Operation-based CRDTs (Commutative Replicated Data Types, or CmRDTs): Instead of shipping the full state, replicas broadcast the update operations themselves. To ensure convergence, these operations must be commutative, meaning they can be applied in any order and still yield the same result. This approach is more bandwidth-efficient but often requires more stringent guarantees from the network layer, such as ensuring that operations are not dropped or duplicated.

While CRDTs are successfully used in collaborative editing software and distributed databases, their application to complex game states is an active area of research [9]. A primary challenge is that many game-world interactions are inherently non-commutative; for example, a Move operation followed by a Shoot operation has a different outcome than the reverse. This creates a significant "impedance mismatch" between the clean, algebraic properties required by simple CRDTs and the messy, stateful, and order-dependent nature of game logic. Applying CRDTs to gaming therefore requires either carefully designing game mechanics to be commutative or developing more complex data structures and algorithms that can handle these ordering constraints, which can erode the initial simplicity of the model. The complexity is not eliminated; it is moved from the network protocol into the data modeling of the game itself.

2.4 Case Studies of Networking in Practice

To bridge the gap between theory and implementation, this section examines the networking models of influential, real-world games. By analyzing seminal titles, we can see how the previously discussed concepts are applied to solve practical challenges. We will dissect Valve's Source Engine as a canonical example of client-side prediction and lag compensation, and Blizzard's Overwatch as a modern hybrid that leverages determinism within an authoritative server to achieve high-fidelity competitive play.

2.4.1 Valve's Source Engine: A Canonical Implementation

The networking model of Valve's Source Engine, which powers titles like Half-Life 2 and Counter-Strike: Source, serves as a well-documented and highly influential example of the authoritative client-server architecture [11]. Its documentation, made widely available to the modding and development community, has codified many of the foundational techniques discussed in this chapter. The engine simulates the game in discrete "ticks" on an authoritative server. Clients run prediction for the local player's movement to mask latency, and the server performs lag compensation to ensure fairness in hit registration by rewinding player history. The client also performs interpolation between received server snapshots to produce smooth motion for remote entities, deliberately rendering the world slightly in the past to ensure it always has valid states to interpolate between [11]. The extensive set of user-configurable variables (cl_interp, cl_cmdrate, rate) allows for fine-tuning of the trade-offs between responsiveness and smoothness, exposing the underlying mechanics to the end-user. The Source Engine's networking model represents a robust, battle-tested blueprint for state synchronization in fast-paced first-person shooters.

Figure 2.7: A visual example of server-side lag compensation in the Source Engine. The red hitboxes represent a player's current (client-view) position, while the blue hitboxes show the rewound historical position that the server used for hit detection. This "favor the shooter" approach ensures that shots register based on what the attacker saw.

2.4.2 Overwatch: Determinism in a High-Paced Shooter

Blizzard Entertainment's Overwatch presents a more modern and novel architecture, as detailed in a GDC 2017 presentation by Timothy Ford [5]. While it operates on an authoritative client-server model, it uniquely "leverages determinism to achieve responsiveness and precision" [5]. This represents a hybrid approach that synthesizes principles from both the state-sync and input-sync paradigms. The server runs the simulation at a fixed tick rate (typically 60Hz for competitive play) and is the final authority on game state. However, the simulation logic itself is deterministic. The client is able to run the exact same simulation code as the server.

This design has significant benefits for client-side prediction. Because the client's simulation is identical to the server's, prediction errors are not caused by subtle divergences in physics or logic (e.g., floating-point inaccuracies). Instead, prediction errors arise only from a lack of information, namely, the inputs of other players that have not yet been received. When the server sends a correction, the client can reconcile its state with high confidence, knowing that the underlying simulation logic is sound. This hybrid model aims to achieve the predictive accuracy of a deterministic system within the responsive, cheat-resistant framework of an authoritative server. This signifies a convergence of the two historically separate paradigms, demonstrating that determinism is not an all-or-nothing architectural choice, but can be employed as a powerful tool within a traditional client-server framework to dramatically improve the quality of prediction and reconciliation. The game's netcode also features an "adaptive interpolation delay" system, which attempts to dynamically adjust the rendering buffer based on network conditions to keep gameplay smooth [5].

2.5 Summary and Research Opportunities

The networking techniques surveyed in this chapter represent decades of pragmatic innovation in response to the fundamental challenge of latency. From client-side prediction to deterministic lockstep, from lag compensation to CRDTs, each approach entails distinct trade-offs between responsiveness, consistency, bandwidth, and computational cost. While these techniques have enabled the modern multiplayer gaming ecosystem, they have evolved through engineering necessity rather than systematic analysis, resulting in a fragmented landscape with poorly understood theoretical foundations.

The dominant paradigm of rollback reconciliation exemplifies both the success and limitations of current approaches. By re-simulating gameplay after receiving authoritative updates, it achieves the responsiveness essential for action games while maintaining eventual consistency. However, this comes at a computational cost that scales linearly with network latency: clients must reprocess O(n) frames for each reconciliation event. For mobile devices, cloud gaming platforms, or games with complex physics, this overhead becomes increasingly problematic. Alternative approaches address specific limitations but introduce new constraints: deterministic lockstep forces all players to experience worst-case latency, CRDTs struggle with non-commutative game operations, and hybrid architectures require substantial engineering investment.

These observations reveal critical gaps in our understanding and implementation of game state synchronization. The lack of formal frameworks makes it difficult to rigorously compare techniques or predict their behavior. The computational burden of re-simulation remains unaddressed despite widespread adoption. The rigid trade-offs between existing approaches suggest unexplored regions in the design space. This motivates the central question of this thesis: can we develop a reconciliation method that preserves the responsiveness of client-side prediction while eliminating the computational overhead of re-simulation?

The following chapter will formalize these concepts mathematically, providing the foundation for introducing algebraic server reconciliation, a novel method that leverages mathematical properties of state changes to achieve direct correction without repeated simulation.

3 Network model

To rigorously discuss and analyze networking techniques for online games, particularly those involving state synchronization and reconciliation, it is essential to first establish a clear and formal understanding of the underlying concepts. This chapter lays the theoretical groundwork by defining fundamental components of a game from a networking perspective, such as game state, state progression, and the roles of clients and the server. Building upon these definitions, we will then explore several established network models, from naive approaches to more sophisticated techniques like client-side prediction and server reconciliation. This exploration will highlight the challenges inherent in achieving responsive and consistent gameplay in a networked environment. Finally, this chapter will introduce the core theoretical concepts behind delta states and culminate in the formal definition of algebraic server reconciliation, the novel method investigated in this thesis.

3.1 Overview

We define a game world using a game state S, an initial game state $s_0 \in S$ and a progression function $f : (S, I) \to S$ as $G = (S, s_0, f)$ where I is some external input. A game state at time t can be progressed using some input $i_t \in I$ to time $t + 1$ using the progression function: $s_{t+1} = f(s_t, i_t)$. We can combine the input and state to a frame $r_t = (s_t, i_t)$.

We say that a machine $m \in M$ is running the game G with state s_t at some time t. Clients are machines $c \in C \subseteq M$ which contribute input $i_t^c \in I^c$ to form the whole input $i_t^c \in i_t$. We assume that one machine is designated as the host and all clients can only communicate with the host. The state on the host is considered the truth if states between machines differ.

Example of a formal game definition

To illustrate the concepts introduced thus far, we consider a simplified game inspired by the mechanics of “agar.io”. In this game, the world is viewed from a top-down perspective where each player is represented as a circle. While the original game features collision and consumption mechanics, our simplified version focuses solely on movement to clearly demonstrate the networking concepts. Each player is assigned a unique identifier to distinguish them within the game state.

We formally define this “top-down movement game” as follows:

Vector	$V \subseteq \mathbb{R} \times \mathbb{R}$
Player	$P = V$
State	$S \subseteq \mathrm{id} \times P$
Input for a single player	$I_P \subseteq \mathbb{B}^4$
Input	$I \subseteq \mathrm{id} \times I_P$

The input representation captures the four directional keys that can be pressed during a single frame. Assuming standard addition operations on boolean values and vectors, we define the progression function:

$$
\begin{aligned}
f^{\text{transform}}\big((i_{\text{up}}, i_{\text{down}}, i_{\text{left}}, i_{\text{right}})\big) &= \big((i_{\text{down}} - i_{\text{up}}, i_{\text{right}} - i_{\text{left}})\big) \\
f^{\text{find}}(\mathrm{id}, S) &= x_P, (\mathrm{id}, x_P) \in S \\
f(s, i) &= \big\{\big(\mathrm{id}, s_p + f^{\text{transform}}\big(f^{\text{find}}(\mathrm{id}, i)\big)\big) \mid \big(\mathrm{id}, s_p\big) \in s\big\}
\end{aligned}
$$

Each player consists of a two-dimensional position vector. The game state comprises the set of all player positions indexed by their identifiers. The progression function operates in two stages: first, it extracts the appropriate input for each player from the combined input set, then it transforms these boolean directional inputs into velocity vectors that update the player positions.

Consider the following concrete example with two players:

$$
\begin{aligned}
s_0 &= \{(p_0, (0, 0)), (p_1, (5, 5))\} \\
i_0 &= \{(p_0, (1, 0, 0, 1)), (p_1, (0, 0, 1, 0))\} \\
f(s_0, i_0) = s_1 &= \{(p_0, (0 + 1, 0 + 1)), (p_1, (5 - 1, 5))\} \\
&= \{(p_0, (1, 1)), (p_1, (4, 5))\}
\end{aligned}
$$

In this example, player p_0 moves diagonally (up and right) while player p_1 moves left, demonstrating how discrete boolean inputs translate into continuous position updates.

Example of client-host architecture

Consider a multiplayer session with two clients c_0 and c_1 connected to a host h. Using our previously defined movement game, we examine how inputs from multiple clients are processed and synchronized through the central host.

Client c_0 generates input $i_t^{c_0} = (1, 0, 0, 1)$, indicating upward and rightward movement. Simultaneously, client c_1 produces input $i_t^{c_1} = (0, 0, 1, 0)$, representing leftward movement. For multiplayer functionality to work correctly, each client must be aware of all other players' inputs, necessitating a mechanism for input distribution.

Our architecture employs a central host model for this distribution. Following the principle that client-side computation cannot be trusted for security reasons, only the host possesses the authority to evaluate the progression function and determine the canonical game state. This does not preclude clients from performing local predictions for responsiveness, as we shall explore in later sections.

The communication flow proceeds as follows: both clients continuously transmit their inputs to the host h. Upon receiving these inputs, the host updates its authoritative state using the progression function:

$$
s_1 := f(s_0, \{(p_0, i_t^{c_0}), (p_1, i_t^{c_1})\})
$$

The host then distributes the updated state to all clients. While various optimization strategies exist for this distribution (such as sending only state deltas or relevant subsets), the simplest implementation transmits the complete state s_1 to ensure all clients maintain a consistent view of the game world.

3.1.1 Naive model

We have one host h and n clients c_i. Each client c has a round trip time of t_c^r. We define the round trip time as the sum of the send t_c^s and receive t_c^e latency $t_c^r = t_c^s + t_c^e$. At each tick, the host sends its state s_t to each client and each client sends input i_t^c.

Both the client and the host run with a fixed time interval t^i which is usually smaller than t_c^r. Therefore, the host cannot wait with processing all input i_t^c and will use the most recent input instead. This means that each client will only be able to react with input i_t^c to state $s_{t-t_c^r}$. We call this the naive model.

Example of naive model networking

To demonstrate the temporal challenges inherent in the naive model, we trace through a concrete execution scenario using our movement game. Consider two clients with asymmetric network conditions:

- Client c_0: round trip time $t_{c_0}^r = 50$ ms (25 ms send, 25 ms receive)
- Client c_1: round trip time $t_{c_1}^r = 200$ ms (100 ms send, 100 ms receive)
- Fixed tick interval: $t^i = 50$ ms

Starting with initial state $s_0 = \{(p_0, (0,0)), (p_1, (5,5))\}$, the host broadcasts this state at tick 0. The following table traces the first 10 ticks of execution:

Tick	Host State	c_0 Sees	c_1 Sees	c_0 Input	c_1 Input	Inputs at Host
0	$(0,0),(5,5)$	—	—	—	—	None
1	$(0,0),(5,5)$	—	—	—	—	None
2	$(0,0),(5,5)$	$(0,0),(5,5)$	—	right	—	None
3	$(0,1),(5,5)$	$(0,0),(5,5)$	$(0,0),(5,5)$	right	—	c_0: right
4	$(0,2),(5,5)$	$(0,1),(5,5)$	$(0,0),(5,5)$	right	up	c_0: right
5	$(0,3),(5,5)$	$(0,2),(5,5)$	$(0,1),(5,5)$	right	up	c_0: right
6	$(0,4),(6,5)$	$(0,3),(5,5)$	$(0,2),(5,5)$	right	up	Both inputs
7	$(0,5),(7,5)$	$(0,4),(6,5)$	$(0,3),(5,5)$	right	up	Both inputs
8	$(0,6),(8,5)$	$(0,5),(7,5)$	$(0,4),(6,5)$	right	up	Both inputs
9	$(0,7),(9,5)$	$(0,6),(8,5)$	$(0,5),(7,5)$	right	up	Both inputs

The execution reveals two critical issues with the naive model. First, examining the input delay: client c_0 receives the initial state at tick 2 and immediately sends a “right” input. This input reaches the host at tick 3, where it gets processed and updates player p_0 to position $(0,1)$. However, c_0 doesn’t observe this change until tick 4, a full round trip time of 50 ms after sending the input.

Similarly, client c_1 receives the initial state at tick 3, sends an "up" input at tick 4, but the host doesn't process it until tick 6 due to the 100 ms transmission delay. Client c_1 finally sees their first movement at tick 8, experiencing a 200 ms delay between action and feedback.

The second, more subtle issue is the competitive unfairness in control responsiveness. Client c_0 observes their input taking effect at tick 4, allowing them to make informed decisions about their next move by tick 5. If they realize they're moving in the wrong direction, they can correct course with only a 50 ms feedback loop. In contrast, client c_1 doesn't see their first movement until tick 8, a 200 ms delay that severely hampers their ability to make tactical adjustments.

Consider a scenario where both players accidentally move toward a hazard: client c_0 can recognize and correct their mistake within 2 ticks, while client c_1 continues blind movement for 4 ticks before seeing any feedback. This creates a fundamental inequality where the lower-latency player has superior control precision, faster error correction, and more responsive gameplay, advantages that compound over time and cannot be overcome through player skill alone.

3.1.2 Fair naive model

Each client c can only respond to a state s_t with an input arriving at the host t_c^r time later. Since t_c^r is different for every player, this is not fair.

We define a fair model as: given a fixed delay δ, every client can respond to the same state s_t with input $i_{t+\delta}^c$. We can make the previous model fair by only processing input on the host, once we received input from all players. Every player is able to react to state s_t with the same delay of $\delta = \max_i t_i^r$. Intuitively, we artificially delay what each player can see to the slowest player. We call this the fair naive model.

Many games are based on reaction, doing an action in response to an event. A major problem with the (fair) naive model is that player inputs in reaction to a present state will only be applied to a future state on the host.

Example of fair naive model networking

The fair naive model addresses the competitive inequality of the naive model by synchronizing all players to the slowest connection. Using the same network configuration and initial state:

- Client c_0: round trip time $t_{c_0}^r = 50$ ms (25 ms send, 25 ms receive)
- Client c_1: round trip time $t_{c_1}^r = 200$ ms (100 ms send, 100 ms receive)
- Fixed tick interval: $t^i = 50$ ms

Starting with initial state $s_0 = \{(p_0, (0,0)), (p_1, (5,5))\}$, the host broadcasts at tick 0. The key difference is that the host now buffers inputs until all players' commands for a given game state have arrived:

Tick	Host State	c_0 Sees	c_1 Sees	c_0 Input	c_1 Input	Host Buffers
0	$(0,0),(5,5)$	—	—	—	—	Empty
1	$(0,0),(5,5)$	—	—	—	—	Empty
2	$(0,0),(5,5)$	$(0,0),(5,5)$	—	right	—	Empty
3	$(0,0),(5,5)$	$(0,0),(5,5)$	$(0,0),(5,5)$	right	—	c_0: right
4	$(0,0),(5,5)$	$(0,0),(5,5)$	$(0,0),(5,5)$	right	up	c_0: right

5	$(0,0),(5,5)$	$(0,0),(5,5)$	$(0,0),(5,5)$	right	up	c_0: right
6	$(0,1),(6,5)$	$(0,0),(5,5)$	$(0,0),(5,5)$	right	up	Both active
7	$(0,2),(7,5)$	$(0,0),(5,5)$	$(0,0),(5,5)$	right	up	Both active
8	$(0,3),(8,5)$	$(0,1),(6,5)$	$(0,0),(5,5)$	right	up	Both active
9	$(0,4),(9,5)$	$(0,2),(7,5)$	$(0,1),(6,5)$	right	up	Both active

The fair naive model fundamentally changes how the host processes inputs. When client c_0's input arrives at tick 3, the host does not immediately apply it. Instead, it stores this input in a buffer and continues waiting. The host maintains the game state at $(0,0),(5,5)$ for ticks 3, 4, and 5, despite having valid input from c_0. Only at tick 6, when client c_1's input finally arrives after its 100 ms journey, does the host process both inputs simultaneously.

This buffering strategy creates perfect fairness in reaction timing. Both clients receive the initial state and have the opportunity to respond before any inputs are processed. When the host finally applies inputs at tick 6, it creates state $(0,1),(6,5)$ where both players have moved. Client c_0 sees this result at tick 8, while client c_1 sees it at tick 9. Critically, both players observe their inputs taking effect relative to the same initial state, eliminating the advantage that lower latency previously provided.

However, this fairness comes at a significant cost to responsiveness. Client c_0, who previously enjoyed a 50 ms feedback loop, now experiences a 200 ms delay between input and observation, matching the slowest player. The buffering essentially forces all players to experience the worst-case latency in the session. In our example, client c_0 sends their input at tick 2 but doesn't see the result until tick 8, a delay of 300 ms total.

This artificial throttling makes the game feel sluggish for players with good connections, potentially driving them away from servers with high-latency participants. The fair naive model thus trades individual responsiveness for competitive balance, a compromise that works poorly for fast-paced action games where immediate feedback is crucial to the gameplay experience.

3.1.3 Lockstep

In the fair naive model, the host is applying old inputs because the interval time is fixed and usually faster than t_c^r. We can loosen this requirement and assume a dynamic interval time. Players only send input i_t^c, after receiving s_t. The host only progresses to state s_{t+1} when all i_t^c have arrived. We now have a fair game while giving players the most recent information to react to.

This model is known as the lockstep model and popular in strategy games, where latency or a longer interval are not a big problem.

Example of lockstep model

The lockstep model ensures perfect synchronization by pausing the entire simulation after each state update until all clients have received and responded. Using the same network configuration:

- Client c_0: round trip time $t_{c_0}^r = 50$ ms (25 ms send, 25 ms receive)
- Client c_1: round trip time $t_{c_1}^r = 200$ ms (100 ms send, 100 ms receive)
- Fixed tick interval: $t^i = 50$ ms

Starting with initial state $s_0 = \{(p_0, (0,0)), (p_1, (5,5))\}$, the host initiates the first lockstep cycle at tick 0:

Tick	Host State	c_0 Sees	c_1 Sees	c_0 Input	c_1 Input	Host Action
0	$(0,0), (5,5)$	—	—	—	—	Broadcast s_0
1	Waiting	$(0,0), (5,5)$	—	right	—	Wait for all
2	Waiting	Sent right	$(0,0), (5,5)$	—	up	Wait for all
3	Waiting	Waiting	Sent up	—	—	Wait for all
4	$(0,1), (6,5)$	Waiting	Waiting	—	—	Process inputs
5	$(0,1), (6,5)$	$(0,1), (6,5)$	Waiting	right	—	Broadcast s_1
6	Waiting	Sent right	$(0,1), (6,5)$	—	up	Wait for all
7	Waiting	Waiting	Sent up	—	—	Wait for all
8	$(0,2), (7,5)$	Waiting	Waiting	—	—	Process inputs
9	$(0,2), (7,5)$	$(0,2), (7,5)$	Waiting	right	—	Broadcast s_2

The lockstep model operates in discrete, synchronized cycles that guarantee perfect consistency across all clients. Each cycle follows a rigid pattern: the host broadcasts a state, waits for all clients to receive it, waits for all clients to send their inputs based on that state, and only then processes those inputs to generate the next state. This creates natural synchronization points where the entire game pauses until the slowest participant catches up.

In our execution, the first cycle begins at tick 0 when the host broadcasts s_0. Client c_0 receives this state at tick 1 and immediately sends a “right” input that reaches the host at tick 2. However, the host cannot proceed because client c_1 only receives the state at tick 2 and sends an “up” input that doesn’t arrive until tick 4. Only then can the host process both inputs together, updating the state to $(0,1), (6,5)$ and beginning the second cycle at tick 5.

The lockstep model achieves perfect fairness and eliminates all prediction or synchronization issues. Every client sees exactly the same game state before making decisions, and all inputs are processed simultaneously with no player gaining an advantage from lower latency. This makes it ideal for turn-based strategy games or scenarios where absolute consistency matters more than responsiveness.

However, the cost is severe: the game effectively runs at the speed of the slowest connection. With client c_1’s 200 ms round trip time, the game can only update every 4 ticks (200 ms), reducing the effective update rate from 20 Hz to 5 Hz. This dramatic slowdown makes the lockstep model unsuitable for any real-time game. Furthermore, if any client experiences a temporary network spike or packet loss, the entire game freezes for all players until that client recovers. The model essentially chains all players to the worst network conditions present in the session, making it impractical for modern action games where fluid, responsive gameplay is essential.

3.1.4 Player observation limitation

We are bound in the time a player is able to react to other players’ inputs. One player has to first send their inputs and another player can only then receive them. Any changes to the state from player c' will never be visible to another player c faster than $t^e_c + t^s_{c'}$.

When the interval rate is fixed to a time $\delta < t_c^e + t_{c'}^s$, which is usually the case, a client c will already have made an input i_{t+1}^c without there being a possibility of receiving input $i_t^{c'}$ from client c'. Therefore according to our definition, games with a fixed interval cannot be fair.

Example of player observation limitation

To illustrate this fundamental constraint, consider our two clients attempting to react to each other's movements. Assume optimal conditions with immediate processing and no artificial delays:

- Client c_0: send 25 ms, receive 25 ms
- Client c_1: send 100 ms, receive 100 ms
- Fixed tick interval: $t^i = 50$ ms

Suppose at tick 0, player p_1 suddenly moves to block a pathway at position $(3, 5)$. Client c_1 sends this input immediately, taking 100 ms to reach the host. The host processes it instantly and broadcasts the update, which takes another 25 ms to reach client c_0. Therefore, client c_0 cannot possibly see player p_1's blocking move until 125 ms after c_1 initiated it.

This 125 ms minimum reaction window exists regardless of the networking model employed. During this time, if the tick interval is 50 ms, client c_0 will have already made at least two movement decisions without any possibility of knowing about the blocked path. If c_0 was moving toward position $(3, 5)$, they would continue moving toward it for at least two ticks before the blocking becomes visible.

This limitation becomes particularly problematic in competitive scenarios requiring split-second reactions. In a fighting game where a 100 ms reaction time separates amateur from professional players, this network-imposed delay makes true real-time interaction impossible. The only way to reduce this limitation is through better network infrastructure (lower latency), not through improved networking models. This physical constraint fundamentally shapes how we must design networked games, leading to the prediction and reconciliation techniques discussed in subsequent sections.

3.1.5 Client side prediction

In the (fair) naive model, a client will only be able to apply their input i_t^c to a state $s_{t-t_c^r}$. Therefore, a client sees responses to their own input with a latency of t_c^r.

While we previously showed an absolute limitation in seeing other players' inputs, there is no such limitation on seeing state changes to your own inputs. We define a prediction function $f^p : I^c \times S \to S$ which only progresses the state based on one single client input i_t^c. We write $p_{t,d}^c$ for a client c, predicting from timestamp t, d ticks to the future. Each client now receives state s_t and directly applies their input to get predicted state $p_{t,1}^c = f^p(i_t^c, s_t)$. Since responses to i_t^c will be received in $s_{t+t_c^r}$, we predict the state up to $p_{t,t_c^r}^c$. This method is called client-side prediction and used in most modern games as it provides the client with instantaneous feedback - making gameplay fluent.

The difficulty with this method is to apply new incoming host state s_{t+1} correctly. Given a client predicted $p_{t,t_c^r}^c$, it will receive a response to input i_t^c in at least t_c^r time as the state $s_{t+t_c^r}$. A previous state change arriving as s_{t+1} would override our predicted state $p_{t,t_c^r}^c$, making the game snap, caused by the time difference. The original solution to this problem is to not replace our

current state but define a partial application function $f^{\text{partial}} : S \times S \to S$ which compares the predicted state with the actual state and only replaces diverging parts. During normal operation, the prediction will be close enough causing only rare desynchronizations, which will be visually visible as snapping. We observe that some parts of the state will always diverge, if state changes are caused by other players or randomness.

One very popular game that is using a slightly modified variation of this method is Minecraft. In Minecraft, the game does not send its direct inputs, but its predicted state [12]. Most important parts of the state to be predicted are breaking blocks and all character movement. The partial application does not happen at client-side, but on host-side. Therefore, the host validates if the incoming input looks good enough and doesn't resimulate it if not necessary. When detecting diverging state, the host sends state corrections. Unfortunately, this limits anti-cheat capabilities to how sophisticated the validation in the partial application function is [12].

One major problem with client-side reconciliation is that finding a good partial application function can be difficult. The reason is that we are looking at divergences between s_{t+1} and p^c_{t,t^r_c}, therefore a difference in time of $t^r_c - 1 \approx t^r_c$. Therefore, even a perfect prediction will diverge on the scale of the round trip time.

One alternative approach would be to not compare the current state for divergence, but memorize past states. Given an interval time of t^i, the client needs to memorize $n_{\text{predict}} = t^r_c / t^i$ game states, which can be partial states. This solution unfortunately does not solve the snapping when divergences are found, since it forces the client from a predicted state in time $t + t^r_c$ to $t + 1$.

Example of client-side prediction

To demonstrate client-side prediction in our movement game, we first define a prediction function that allows a client to speculatively apply their own inputs without waiting for server confirmation. Given that we define clients as identifiers, we can formalize the prediction function as follows:

$$
\begin{aligned}
f^{\text{transform}}\big((i_{\text{up}}, i_{\text{down}}, i_{\text{left}}, i_{\text{right}})\big) &= \big((i_{\text{down}} - i_{\text{up}}, i_{\text{right}} - i_{\text{left}})\big) \\
f^{\text{find}}(\text{id}, S) &= x_P, \quad x_P \in \{(\text{id}', x') \in S \mid \text{id} = \text{id}'\} \\
f^{\text{update}}\big(\text{id}, s_p, S\big) &= \big\{\big(\text{id}, s_p\big)\big\} \cup \{(\text{id}', x') \in S \mid \text{id} \neq \text{id}'\} \\
f^p(i_c, \text{id}, s) &= f^{\text{update}}\big(\text{id}, f^{\text{find}}(\text{id}, s) + f^{\text{transform}}(i_c), s\big)
\end{aligned}
$$

The prediction function f^p takes a client's input i_c, the client's identifier, and the current state, then returns an updated state where only the predicting client's position has changed. Other players remain at their last known positions from the server's authoritative state.

Consider a scenario with client c_0 controlling player p_0, with a round trip time of 100 ms (50 ms each direction) and a tick interval of 25 ms. Without prediction, the client would experience a 100 ms delay between pressing a key and seeing their character move. With client-side prediction, the response becomes instantaneous:

Tick	Server State	Client Receives	Client Input	Predicted State	Client Display
0	$(0,0),(5,5)$	—	—	—	$(0,0),(5,5)$
1	$(0,0),(5,5)$	—	—	—	$(0,0),(5,5)$
2	$(0,0),(5,5)$	$(0,0),(5,5)$	right	$(1,0),(5,5)$	$(1,0),(5,5)$
3	$(0,0),(5,5)$	$(0,0),(5,5)$	right	$(2,0),(5,5)$	$(2,0),(5,5)$

4	$(1,0),(5,5)$	$(0,0),(5,5)$	right	$(3,0),(5,5)$	$(3,0),(5,5)$
5	$(2,0),(5,5)$	$(0,0),(5,5)$	right	$(4,0),(5,5)$	$(4,0),(5,5)$
6	$(3,0),(6,5)$	$(1,0),(5,5)$	stop	$(4,0),(5,5)$	$(4,0),(5,5)$
7	$(3,0),(7,5)$	$(2,0),(5,5)$	stop	$(4,0),(5,5)$	$(4,0),(5,5)$
8	$(3,0),(8,5)$	$(3,0),(6,5)$	stop	$(3,0),(6,5)$	$(3,0),(6,5)$
9	$(3,0),(9,5)$	$(3,0),(7,5)$	stop	$(3,0),(7,5)$	$(3,0),(7,5)$

At tick 2, the client receives the initial state and immediately begins moving right. Without prediction, the client would display $(0,0)$ until tick 6. With prediction, the movement appears instantaneous: the client displays $(1,0)$ at tick 2, even though the server hasn't processed this input yet. The client continues predicting movement, reaching $(4,0)$ by tick 5.

The critical moment occurs at tick 6 when the client receives the server state $(1,0),(5,5)$. This confirms the client's first input was processed, but also reveals that player p_1 has moved to $(6,5)$ - information the client couldn't predict. At this point, the predicted position $(4,0)$ must be reconciled with the server's authoritative position $(1,0)$ plus the three additional predictions made since then.

Notice how the prediction creates a smooth experience for the controlling player: they press right and immediately see movement. However, it also introduces divergence between the predicted and authoritative states. In our example, the client predicted player p_0 at position $(4,0)$ at tick 6, while the server only had them at $(3,0)$. This divergence of one position unit represents the unconfirmed prediction still in transit to the server.

The prediction function must carefully handle state updates to avoid "snapping" - sudden position corrections that break immersion. When the server state arrives at tick 8 showing $(3,0),(6,5)$, the client has predicted $(4,0)$ but stopped sending input at tick 6. The reconciliation process must recognize that position $(3,0)$ represents the final authoritative position after processing all inputs, allowing the display to smoothly converge to the correct state.

This example demonstrates the fundamental trade-off of client-side prediction: immediate responsiveness at the cost of temporary divergence from the authoritative state. The prediction allows fluid gameplay even with 100 ms latency, transforming what would be a sluggish experience into one that feels instantaneous. However, it requires sophisticated reconciliation mechanisms to handle the inevitable corrections when predicted and authoritative states diverge, particularly when multiple players' actions interact in ways the client cannot predict locally.

3.1.6 Server reconciliation

When merging an update s_{t+1} with a diverged predicted state p^c_{t,t^r_c}, we previously overrode the state with state s_{t+1}. This skip from $t+t^r_c$ to $t+1$ causes snapping. What we can do alternatively is to memorize inputs $i^c_k : t+1 \leq k \leq t+t^r_c$. We replace our state with s_{t+1} and then apply all memorized inputs to get an alternative predicted state $p^c_{t,t^r_c}{}'$ which would have been reached if the divergence didn't happen. Since we now have a state in time $t+t^r_c$, we do not have snapping behavior. This method is called rollback server reconciliation and is one of the most fundamental techniques in modern games.

Server reconciliation is the process of combining a past received state s_t with a present predicted state p^c_{t,t^r_c}. Our partial application function is a form of server reconciliation. The goal is to fix divergences in state without the client noticing.

Unfortunately, since we have to rollback to the last received state, we must memorize all predictions we made. In the worst case, we memorize the whole s_t. When receiving s_t, we apply the prediction function n_{predict} times. Therefore an input i_t^c is predicted for the first time at $t - n_{\text{predict}}$ and the last time at t. So each input is processed by the client n_{predict} times. This means that especially clients with longer ping have to do more processing, increasing with a smaller tick interval.

Example of server reconciliation

To illustrate server reconciliation with rollback, we examine how a client corrects prediction errors when receiving authoritative updates from the server. Consider our movement game with client c_0 controlling player p_0, with a round trip time of 100 ms (50 ms each direction) and a tick interval of 25 ms. This means the client must predict 4 ticks ahead ($n_{\text{predict}} = \frac{100}{25} = 4$) to maintain responsive gameplay.

The client memorizes all inputs sent to the server that haven't been acknowledged yet. When a server update reveals a prediction error, the client rolls back to the last confirmed state and re-simulates all memorized inputs. This process is visualized in Figure 3.1.

Consider the following execution, where player p_1 (controlled by another client) moves in a way the client cannot predict:

Tick	Server State	Client Receives	Client Input	Memorized	Predicted State	Display
0	$(0,0),(5,5)$	—	—	[ ]	—	$(0,0),(5,5)$
1	$(0,0),(5,5)$	—	—	[ ]	—	$(0,0),(5,5)$
2	$(0,0),(5,5)$	$(0,0),(5,5)$	right	right	$(1,0),(5,5)$	$(1,0),(5,5)$
3	$(0,0),(5,5)$	$(0,0),(5,5)$	right	right, right	$(2,0),(5,5)$	$(2,0),(5,5)$
4	$(1,0),(5,5)$	$(0,0),(5,5)$	up	right, right, up	$(2,1),(5,5)$	$(2,1),(5,5)$
5	$(2,0),(5,5)$	$(0,0),(5,5)$	up	right, right, up, up	$(2,2),(5,5)$	$(2,2),(5,5)$
6	$(2,1),(4,5)$	$(1,0),(5,5)$	stop	right, up, up, stop	**Rollback**	—
*	—	—	—	—	$(1,0),(5,5)$	State after rollback
*	—	—	—	—	$(2,0),(5,5)$	Re-sim: right
*	—	—	—	—	$(2,1),(5,5)$	Re-sim: up
*	—	—	—	—	$(2,2),(5,5)$	Re-sim: up
*	—	—	—	[stop]	$(2,2),(5,5)$	$(2,2),(5,5)$
7	$(2,2),(3,5)$	$(2,0),(5,5)$	stop	up, up, stop, stop	**Rollback**	—
*	—	—	—	—	$(2,0),(5,5)$	State after rollback
*	—	—	—	—	$(2,1),(5,5)$	Re-sim: up

*	—	—	—	—	$(2,2),(5,5)$	Re-sim: up
*	—	—	—	—	$(2,2),(5,5)$	Re-sim: stop
7	—	—	—	[stop]	$(2,2),(5,5)$	$(2,2),(5,5)$
8	$(2,2),(2,5)$	$(2,1),(4,5)$	left	stop, stop, left	**Rollback**	—

The table traces the execution of server reconciliation with rollback over 8 ticks, showing how the client maintains responsiveness through prediction while handling authoritative corrections from the server. The "Memorized" column tracks unacknowledged inputs retained for re-simulation, while rows marked with asterisks (*) detail the internal rollback and re-simulation steps occurring within a single tick when discrepancies are detected.

The critical reconciliation occurs at tick 6. The client has predicted player p_0 at position $(2,2)$ based on its "right, right, up, up" sequence, but the server state $(1,0),(5,5)$ reveals that only the first "right" input has been processed server-side. The rollback process, illustrated in Figure 3.1, begins by reverting to the server's authoritative state $(1,0),(5,5)$. From this baseline, the client reconstructs its predicted state by re-applying all unacknowledged inputs in sequence: the second "right" transforming $(1,0)$ to $(2,0)$, two "up" inputs advancing through $(2,1)$ to $(2,2)$, and finally "stop" maintaining the position. The acknowledged first "right" is then removed from the memorized buffer.

Following re-simulation, the client achieves state $(2,2),(5,5)$, where player p_0's position matches the original prediction. This outcome is characteristic when prediction errors stem solely from other players' unpredictable movements. The reconciliation at tick 8 presents a more complex case where player p_1 has moved to $(4,5)$, introducing a persistent divergence.

The computational implications of this approach become evident when considering the frequency and scope of re-simulation. Each server update necessitates re-processing all unacknowledged inputs, which in our configuration amounts to 4 inputs per reconciliation event. For clients experiencing higher latency, such as 200 ms round trip time, the re-simulation burden increases to 8 or more inputs per server update. This computational overhead scales linearly with latency, presenting a significant performance challenge that motivates the investigation of alternative reconciliation methods, particularly our proposed algebraic reconciliation approach which aims to eliminate the need for repeated re-simulation entirely.

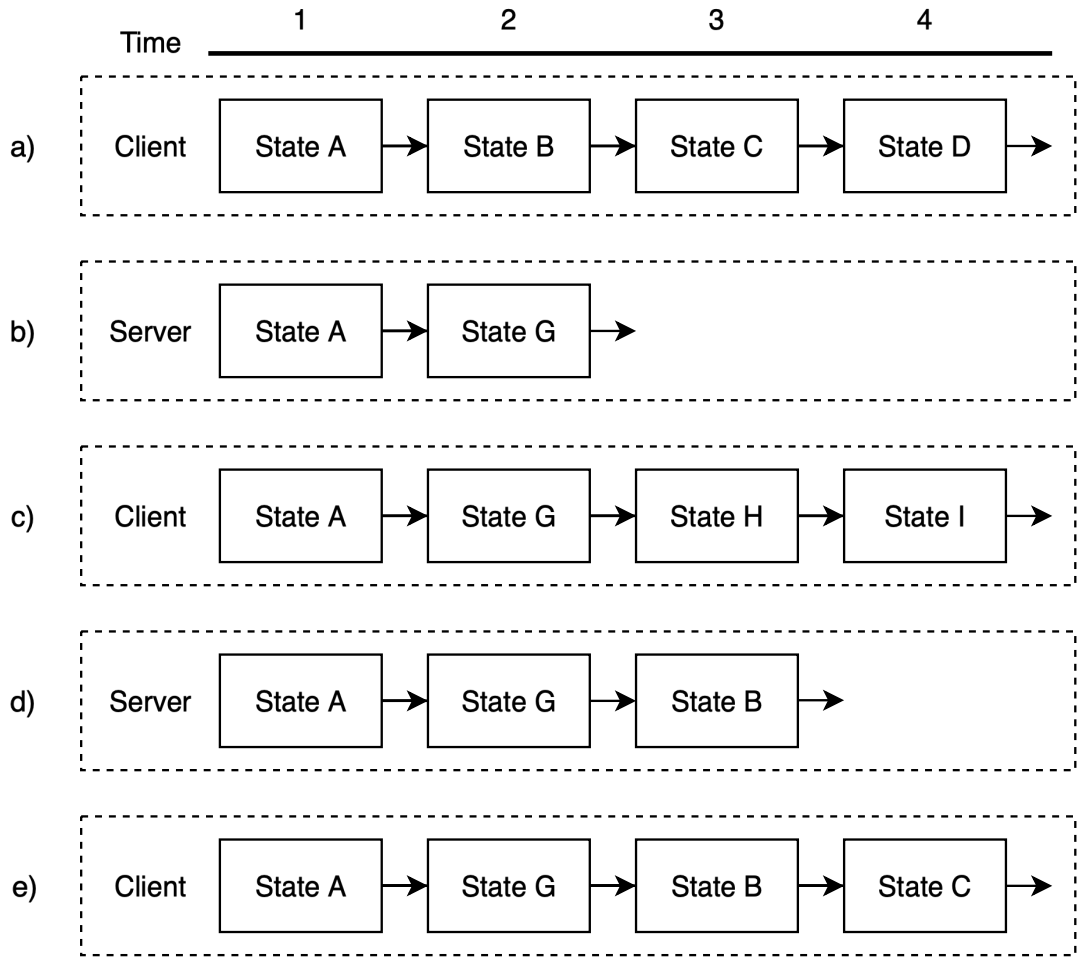

Figure 3.1: Illustration of the rollback netcode process. (a) The client's initial predicted state progression. (b) The server's authoritative state diverges (State G instead of B). (c) Upon receiving the authoritative State G, the client rolls back to State A, applies State G, and then re-simulates its subsequent inputs to reach States H and I. (d) The server again processes inputs and generates a new authoritative State B, which differs from the client's expectation (G). (e) The client performs another rollback, reverting to a prior known good state, applying the server's authoritative State B, and re-simulating all inputs to reach State C. This demonstrates how rollback continuously adjusts the client's timeline by repeatedly re-computing future states (e.g., the state at time point 4 is computed multiple times) based on new authoritative information, ensuring eventual consistency.

3.1.7 Delta State

The host is sending the whole state s_t each tick. This means we will send redundant data as the state rarely changes completely. We define delta states $\Delta s_t \in \Delta S$ as changes in state which can be applied using a delta application function $f^A : S \to \Delta S \to S$. We define an alternative progression function $f^\Delta : I \to S \to \Delta S$ returning a delta instead of the whole state and write $f^\Delta(i_t, s_t) = \Delta s_t$. We can derive the original progression function as $f(i_t, s_t) = f^A\big(s_t, f^\Delta(i_t, s_t)\big)$, therefore all our previous results hold true when using delta states. The other way around, we can convert any normal state to delta state given progression function f and state S by defining

$f^A = f$, $\Delta S = S$ and $f^A(\cdot, s_t) = s_t$. This means, that in practice we can always implement delta state and use normal state on top.

The host will each tick send Δs_{t+1} to the clients. In the (fair) naive or lockstep model, a client has currently loaded a state s_t and can simply use the delta application function to progress the state. Using rollback reconciliation, the client can use the delta application function after the rollback to s_t and predict the actual state afterwards. If delta state can be used with the partial application reconciliation depends on the state and cannot be generalized because it is not possible to get back to a state s_t.

Example of delta state

To illustrate delta states, we extend our movement game to represent state changes rather than absolute positions. Instead of transmitting complete player positions each tick, we transmit only the changes (deltas) that occurred. This approach significantly reduces bandwidth requirements when players remain stationary or move predictably.

We define delta states using the same vector space as our original game, but now representing position changes:

$$\begin{aligned} &\text{Vector} && V \subseteq \mathbb{R} \times \mathbb{R} \\ &\text{Delta Player} && \Delta P = V \\ &\text{Delta State} && \Delta S \subseteq \text{id} \times \Delta P \end{aligned}$$

The delta application function combines a current state with a delta to produce the next state:

$$\begin{aligned} f^{\text{find}}(\text{id}, s) &= \begin{cases} s_p & \text{if } (\text{id}, s_p) \in s \\ (0,0) & \text{otherwise} \end{cases} \\ f^A(s, \Delta s) &= \left\{ \left(\text{id}, f^{\text{find}}(\text{id}, s) + \Delta p\right) \mid (\text{id}, \Delta p) \in \Delta s \right\} \\ &\quad \cup \left\{ \left(\text{id}, s_p\right) \mid \left(\text{id}, s_p\right) \in s \wedge (\text{id}, \cdot) \notin \Delta s \right\} \end{aligned}$$

The delta progression function generates deltas from inputs rather than complete states:

$$\begin{aligned} f^{\text{transform}}\left(\left(i_{\text{up}}, i_{\text{down}}, i_{\text{left}}, i_{\text{right}}\right)\right) &= \left(\left(i_{\text{down}} - i_{\text{up}}, i_{\text{right}} - i_{\text{left}}\right)\right) \\ f^{\Delta}(i, s) &= \left\{ \left(\text{id}, f^{\text{transform}}\left(i_p\right)\right) \mid \left(\text{id}, i_p\right) \in i \right\} \end{aligned}$$

Consider a concrete example where player p_0 moves diagonally while player p_1 remains stationary:

$$\begin{aligned} s_0 &= \{(p_0, (3,2)), (p_1, (5,5))\} \\ i_0 &= \{(p_0, (1,0,0,1)), (p_1, (0,0,0,0))\} \\ f^{\Delta}(i_0, s_0) &= \{(p_0, (1,1)), (p_1, (0,0))\} = \Delta s_0 \\ f^A(s_0, \Delta s_0) &= \{(p_0, (3,2) + (1,1)), (p_1, (5,5) + (0,0))\} \\ &= \{(p_0, (4,3)), (p_1, (5,5))\} = s_1 \end{aligned}$$

The bandwidth advantage becomes apparent when examining the transmitted data. Without delta states, the server sends the complete state $\{(p_0, (4,3)), (p_1, (5,5))\}$ every tick. With delta states, it sends only $\{(p_0, (1,1))\}$, omitting player p_1 entirely since their position was unchanged (the zero delta $(0,0)$ need not be transmitted).

In our simplified example with just two players possessing only position data, the bandwidth improvement appears modest, reducing transmission from two position vectors to one. However,

this understates the significance of delta states in production games. Consider a typical real-time strategy game with thousands of units, each containing position, health, ammunition, animation states, and numerous other attributes. When only a handful of units move or engage in combat each tick while the vast majority remain idle at their posts, delta states eliminate the need to transmit any data for stationary units. A battlefield with 1000 units where only 50 are active reduces network traffic by 95%, transforming what would be an unmanageable bandwidth requirement into a feasible one. This optimization becomes even more critical for games with complex environmental objects, destructible terrain, or persistent world elements that rarely change but would otherwise require constant retransmission.

This delta representation also naturally supports the identity element required for algebraic operations. The zero vector $(0, 0)$ serves as the identity: $f^{A(s,\emptyset)} = s$, where the empty delta set represents no changes. This property becomes crucial for our algebraic reconciliation method, as it allows us to compose and decompose state changes algebraically.

3.2 Algebraic server reconciliation

The reconciliation methods examined thus far, particularly rollback reconciliation, reveal significant implementation challenges and computational costs that limit their practical applicability. Rollback reconciliation demands deterministic physics simulation to ensure correct results during re-simulation, a requirement that imposes substantial engineering constraints on game development. The simulation must produce bit-identical results across all re-simulation passes, precluding the use of many standard optimizations and requiring careful control over floating-point operations, random number generation, and execution order. Even subtle implementation errors in state management or input ordering can manifest as visible artifacts: jittering movement, rubberbanding effects, or inconsistent collision resolution that degrades the player experience.

Beyond implementation complexity, the computational overhead of rollback reconciliation scales poorly with network latency. As demonstrated in our analysis, clients must re-simulate all unacknowledged inputs with each server update, performing n_{predict} iterations of the progression function per reconciliation event. For clients with higher latency, a common scenario in global multiplayer games, this computational burden becomes prohibitive, particularly when multiple entities require prediction or when the progression function involves expensive physics calculations.

These limitations motivate the development of an alternative reconciliation method that eliminates the need for re-simulation while maintaining the responsiveness benefits of client-side prediction. We now introduce algebraic server reconciliation, a novel approach that leverages the mathematical structure of delta states to compute corrections directly, transforming reconciliation from an iterative re-simulation process into a single algebraic operation. This method not only reduces computational complexity from $O(n_{\text{predict}})$ to $O(1)$ per reconciliation event, but also removes the strict determinism requirement, significantly simplifying implementation and enabling broader applicability across diverse game architectures.

3.2.1 Overview

The theoretical foundation for algebraic server reconciliation draws inspiration from Conflict-Free Replicated Data Types (CRDTs), a well-established technique in distributed systems for achieving eventual consistency without coordination [9]. CRDTs enable multiple replicas in a distributed

system to independently modify shared data while guaranteeing convergence to a consistent state. This guarantee is achieved through carefully designed data structures and operations that possess specific algebraic properties particularly commutativity and associativity allowing updates to be applied in any order while producing the same final result. The merge function in CRDTs serves as a universal conflict resolver, automatically reconciling divergent states without requiring explicit coordination or consensus protocols.

However, the application context for networked games presents a fundamentally different topology than traditional CRDT deployments. In distributed databases or collaborative editing applications, CRDTs operate in a peer-to-peer model where any replica can generate updates and all replicas eventually converge through symmetric merge operations. In contrast, our client-server game architecture establishes an asymmetric relationship: the server maintains authoritative state while clients generate speculative predictions that must ultimately align with server decisions. This architectural distinction enables a more specialized approach than generic CRDTs would provide.

Our method exploits this asymmetry by recognizing that reconciliation is inherently a client-side concern. The server never needs to reconcile, it simply progresses the authoritative state based on received inputs. Only clients face the challenge of aligning their predicted states with authoritative updates. This insight allows us to design a reconciliation mechanism tailored specifically to the temporal dynamics of client-side prediction, where the fundamental problem is correcting a present predicted state based on past authoritative information.

The core mechanism operates through two complementary algebraic operations. First, we define a difference operation that computes the discrepancy between the client's predicted delta and the server's authoritative delta for a given time period. This difference captures the prediction error in the form of a correction delta. Second, we employ a merge operation similar in spirit to CRDT merges but adapted for our temporal context that applies this correction delta to the client's current predicted state. Crucially, because of the inherent time delay between generating predictions and receiving authoritative updates, we cannot directly compare contemporaneous states. Instead, we compute corrections based on historical divergences and apply them to present states, relying on a key assumption about the temporal stability of predictions to ensure convergence.

This approach transforms reconciliation from the computationally expensive process of resimulating multiple frames of game logic into a direct algebraic computation. Where rollback reconciliation must replay n_{predict} frames of potentially complex physics simulation, algebraic reconciliation performs a single difference calculation followed by a single merge operation. The formal properties ensuring convergence under this simplified model will be established through the mathematical framework presented in the following definition.

3.2.2 Definition

An abelian group is a set A, together with an operation $+ : A \times A \to A$ which is associative and commutative. Additionally, it has an identity element $a + e = a$ and for every element a an inverse $a + a' = e$ [6].

We assume a delta state that defines an abelian group with $S = \Delta S$, $e = s_e$ and $+ = f^A$, where f^A satisfies the required properties. We can observe, that we still have advantages over non-delta state, since we do not need to transfer state, when $\Delta s = s_e$.

We define a delta prediction function $f^{\Delta p} : I^c \times S \to \Delta S$ producing a delta $\Delta p^c_{t,d} = f^{\Delta p}\left(i^c_{t+d}, p^c_{t,d}\right)$ instead of the whole state when predicting the next state. Therefore $p^c_{t,d+1} = p^c_{t,d} + \Delta p^c_{t,d}$ and $p^c_{t,d} = s_t + \sum_{k=0}^{d} \Delta p^c_{t,k}$.

The client memorizes $n_{\text{predicted}}$ predicted deltas $\Delta p^c_{t,k} : 0 \leq k \leq t^r_c$. We define a new server reconciliation method as follows: For each incoming Δs_{t+1}, we add a correction update $\epsilon = \Delta s_{t+1} - \Delta p^c_{t,1}$ to our current state $p^c_{t+1,t^r_c} = p^c_{t,t^r_c} + \epsilon$. Assuming that predictions are mostly uncorrelated $\Delta p^c_{t,d} \approx \Delta p^c_{t+1,d-1}$, we can show convergence:

$$
\begin{aligned}
& p^c_{t,t^r_c} + \epsilon \\
&= p^c_{t,t^r_c} + \left(\Delta s_{t+1} - \Delta p^c_{t,0}\right) \\
&= s_t + \sum_{k=0}^{t^r_c} \Delta p^c_{t,k} + \left(\Delta s_{t+1} - \Delta p^c_{t,0}\right) \\
&= \left(s_t + \Delta s_{t+1}\right) + \sum_{k=1}^{t^r_c} \Delta p^c_{t,k} + \left(\Delta p^c_{t,0} - \Delta p^c_{t,0}\right) \\
&= s_{t+1} + \sum_{k=1}^{t^r_c} \Delta p^c_{t,k} \\
&\approx s_{t+1} + \sum_{k=0}^{t^r_c - 1} \Delta p^c_{t+1,k} \\
&= p^c_{t+1,t^r_c - 1}
\end{aligned}
$$

Therefore, we can reach any prediction p^c_{t+d,t^r_c} over time. We call this reconciliation method algebraic server reconciliation.

Example of Abelian Server Reconciliation

To demonstrate algebraic server reconciliation in practice, we must first establish the abelian group structure for our movement game. Unlike previous examples that focused on state progression and prediction, this example requires defining the algebraic operations that enable direct correction computation without re-simulation.

We define an abelian group for our movement game using component-wise vector addition as the group operation. The identity element is the zero vector, and inverse elements are obtained through component-wise negation:

$$
\begin{aligned}
f^{\text{find}}(\text{id}, s) &= \begin{cases} s_p & \text{if } (\text{id}, s_p) \in s \\ (0,0) & \text{otherwise} \end{cases} \\
f^{\text{ids}}(s) &= \{\text{id} \mid (\text{id}, *) \in s\} \\
s + s' &= \left\{\left(\text{id}, f^{\text{find}}(\text{id}, s) + f^{\text{find}}(\text{id}, s')\right) \mid \text{id} \in f^{\text{ids}}(s \cup s')\right\} \\
-s &= \left\{\left(\text{id}, -s_p\right) \mid \left(\text{id}, s_p\right) \in s\right\} \\
e &= \emptyset
\end{aligned}
$$

Consider client c_0 controlling player p_0 with 100 ms round trip time. Starting from state $s_0 = \{(p_0, (0,0)), (p_1, (5,5))\}$, the client sends a "right" input and immediately predicts the delta:

$$\Delta p^c_0 = \{(p_0, (1,0)), (p_1, (0,0))\}$$

The client applies this to display $(1, 0), (5, 5)$ immediately. Due to the 100 ms round trip time, the server will not process this input for 50 ms, and the acknowledgment will not return for another 50 ms. During this time, the client continues predicting, accumulating deltas in its memorized buffer.

When the server's delta finally arrives, it might show:

$$\Delta s = \{(p_0, (1, 0)), (p_1, (2, 0))\}$$

This confirms the client's prediction for p_0, but reveals that p_1 moved right by 2 units, information the client couldn't predict. The algebraic reconciliation computes the correction:

$$\begin{aligned}\varepsilon &= \Delta s - \Delta p_0^c \\ &= \{(p_0, (1, 0)), (p_1, (2, 0))\} - \{(p_0, (1, 0)), (p_1, (0, 0))\} \\ &= \{(p_0, (0, 0)), (p_1, (2, 0))\}\end{aligned}$$

This correction represents exactly what the client failed to predict: player p_1's movement. The client applies this correction directly to its current predicted state without any re-simulation. If the client has since made additional predictions (say, moving up twice for a total displacement of $(0, 2)$), its current displayed state might be $(1, 2), (5, 5)$. Applying the correction:

$$\begin{aligned}s_{\text{corrected}} &= s_{\text{current}} + \varepsilon \\ &= \{(p_0, (1, 2)), (p_1, (5, 5))\} + \{(p_0, (0, 0)), (p_1, (2, 0))\} \\ &= \{(p_0, (1, 2)), (p_1, (7, 5))\}\end{aligned}$$

The key insight is that this correction happens in constant time through a single vector addition, regardless of how many predictions were made since the server state was generated. Rollback reconciliation would need to return to state s_0, apply the server's update to get $(1, 0), (7, 5)$, then re-simulate the two "up" inputs to reach $(1, 2), (7, 5)$. Algebraic reconciliation achieves the same result directly.

This efficiency gain becomes more pronounced with higher latency. A client with 200 ms round trip time would need to maintain and potentially re-simulate 8 ticks of predictions with rollback. With algebraic reconciliation, it still performs just one correction operation. Furthermore, the method tolerates non-deterministic physics, as long as the delta operations maintain their algebraic properties, the correction converges correctly even if the exact simulation differs between client and server.

3.2.3 Limitations

The convergence proof for algebraic server reconciliation relies on a critical assumption: $\Delta p_{t,d}^c \approx \Delta p_{t+1,d-1}^c$. This assumption requires that predictions remain temporally stable, meaning the delta produced by predicting from state s_t at time d should closely match the delta produced by predicting from state s_{t+1} at time $d - 1$. In essence, this assumes that the prediction function exhibits minimal sensitivity to variations in the base state from which predictions originate.

This temporal stability assumption holds well for games where entities operate relatively independently and player control switches frequently between different entities. Consider a real-time strategy game where a player commands hundreds of units. Each unit's movement prediction depends primarily on its own state and the player's direct commands, with minimal influence from other units' positions. When the player issues a move command to unit A, then immediately switches attention to unit B, the predictions for unit A remain consistent regardless of minor corrections to unit B's state. The decorrelation between controlled entities ensures that prediction errors in one entity do not cascade into prediction errors for others.

However, this assumption breaks down in games with strong state dependencies and persistent entity control. Sandbox games exemplify this limitation through their fundamental design philosophy: every entity can interact with any other entity in complex, emergent ways. Consider a scenario where player p_1 pushes player p_0 off a ledge. The falling state fundamentally alters the prediction function's behavior: a falling player accelerates downward each tick, while a grounded player remains stationary. If the client fails to predict the initial push, it continues predicting p_0 as stationary while the server simulates falling. The algebraic correction $\varepsilon = \Delta s - \Delta p$ captures only the position discrepancy for that specific tick, not the change in falling state. Consequently, the client's prediction continues using the wrong dynamics, generating increasingly large errors that the algebraic correction cannot fully resolve. The system perpetually lags behind the true state until the falling motion ceases and both client and server return to similar dynamics.

This limitation extends to state-dependent dynamics. For projectiles, small initial mispredictions in velocity/angle cause trajectory divergence; a positional correction delta does not fix the velocity state, so error compounds until dynamics re-align or the object is removed. Similarly, vehicle physics with momentum, sliding mechanics on ice, or even simple acceleration-based movement all violate the temporal stability assumption to varying degrees.

The severity of these limitations correlates inversely with the frequency of state-altering interactions. Games where velocity changes occur primarily through direct player control exhibit better convergence properties, because the controlling player's inputs naturally align the predicted and actual dynamics. When a player accelerates their character, both client and server process the same acceleration input, maintaining synchronized dynamics despite potential position discrepancies. Problems arise when external forces, especially those from other players or environmental hazards, alter an entity's dynamic state in ways the client cannot predict.

These limitations do not render algebraic reconciliation unusable but rather define its domain of applicability. The method excels in games with discrete, position-based movement, sparse entity interactions, and minimal state-dependent dynamics. For games outside this domain, hybrid approaches that combine algebraic reconciliation for suitable state components with rollback reconciliation for problematic components may offer the best compromise between computational efficiency and correctness.

3.2.4 Abelian group for ECS games

Entity Component System (ECS) architectures have become the predominant method for organizing game state in modern engines. The compositional nature of ECS, where entities are defined by their collection of components rather than through inheritance hierarchies, naturally aligns with the algebraic structures required for our reconciliation method. We now demonstrate how to construct an abelian group for arbitrary ECS-based games, providing a general framework that developers can adapt to their specific implementations.

In an ECS architecture, the game world consists of entities, each possessing a unique identifier. Entities are containers for components, where each component type stores specific data (position, health, inventory, etc.). Systems operate on entities possessing specific component combinations, but for state representation purposes, we focus solely on the entity-component data structure.

We formalize an ECS game state as a nested mapping with multiplicities:

$$\text{World} : \text{EntityID} \to (\text{Entity}, \mathbb{Z})$$
$$\text{Entity} : \text{ComponentTypeID} \to (\text{Component}, \mathbb{Z})$$

The integer multiplicities serve a dual purpose. For entities, positive multiplicity indicates creation or presence, negative multiplicity indicates deletion, and zero represents either modification or an already-deleted entity. For components, multiplicities similarly track additions, removals, and modifications. This multiplicity-based representation naturally encodes state changes as part of the state structure itself.

The abelian group operation combines worlds by summing multiplicities:

$$(W_1 + W_2)[\mathrm{id}_e] = \begin{cases} (E_1 + E_2, m_1 + m_2) & \text{if } \mathrm{id}_e \in W_1 \text{ and } \mathrm{id}_e \in W_2 \\ W_1[\mathrm{id}_e] & \text{if } \mathrm{id}_e \in W_1 \text{ and } \mathrm{id}_e \notin W_2 \\ W_2[\mathrm{id}_e] & \text{if } \mathrm{id}_e \notin W_1 \text{ and } \mathrm{id}_e \in W_2 \\ \text{undefined} & \text{otherwise} \end{cases}$$

Entity addition is defined recursively through component addition:

$$(E_1 + E_2)[\mathrm{id}_c] = \begin{cases} (C_1 + C_2, m_1 + m_2) & \text{if } \mathrm{id}_c \in E_1 \text{ and } \mathrm{id}_c \in E_2 \\ E_1[\mathrm{id}_c] & \text{if } \mathrm{id}_c \in E_1 \text{ and } \mathrm{id}_c \notin E_2 \\ E_2[\mathrm{id}_c] & \text{if } \mathrm{id}_c \notin E_1 \text{ and } \mathrm{id}_c \in E_2 \\ \text{undefined} & \text{otherwise} \end{cases}$$

Component addition must be defined specifically for each component type. For vector positions, we use vector addition. For scalar health values, we use arithmetic addition. For discrete states, we might use replacement semantics where the component with higher absolute multiplicity takes precedence.

Consider a concrete example with two entity types in a simplified game:

$$\begin{aligned} \text{PositionComponent} &= \mathbb{R} \times \mathbb{R} \\ \text{HealthComponent} &= \mathbb{Z} \\ \text{Player} &= \{\text{Position} : \text{PositionComponent}, \text{Health} : \text{HealthComponent}\} \\ \text{Projectile} &= \{\text{Position} : \text{PositionComponent}\} \end{aligned}$$

A world state might be:

$$\begin{aligned} W_0 = \{e_1 : (\{&\text{Position} : ((5,3),1), \\ &\text{Health} : (100,1)\},1), \\ e_2 : (\{&\text{Position} : ((10,7),1)\},1)\} \end{aligned}$$

A delta representing player movement and damage might be:

$$\begin{aligned} \Delta W = \{e_1 : (\{&\text{Position} : ((2,0),1), \\ &\text{Health} : ((-10),1)\},0)\} \end{aligned}$$

Applying this delta yields:

$$\begin{aligned} W_1 = W_0 + \Delta W = \{e_1 : (\{&\text{Position} : ((7,3),1), \\ &\text{Health} : (90,1)\},1), \\ e_2 : (\{&\text{Position} : ((10,7),1)\},1)\} \end{aligned}$$

The identity element is the empty world mapping $\emptyset$, and the inverse of any world is obtained by negating all multiplicities and component values. These operations satisfy the abelian group axioms: closure (combining two ECS states yields another valid ECS state), associativity (the order of combining multiple deltas doesn't matter), commutativity (deltas can be applied in any order), identity (adding an empty delta changes nothing), and inverse (every delta can be undone).

This formalization provides a systematic approach to implementing algebraic reconciliation for any ECS-based game. Developers need only define appropriate addition operations for their specific component types, and the framework handles the compositional structure automatically. The multiplicity-based approach elegantly captures the full lifecycle of entities and components, from creation through modification to deletion, within a unified algebraic structure.

4 Networking Architecture

The goal of a networking system is to provide a seamless interaction with other players inside a game. How to achieve a seamless interaction with other players was investigated in Section 3. The focus of systems architecture is on how to implement these models within the game's underlying system. Consequently, this chapter is of less concern to players and more pertinent to developers. Just as we aim for seamless interaction for players, we also strive for seamless game development for developers. The architecture here is only one possible approach to networking architecture, which will be different depending on needs and game architecture.

The architecture below shows how the networking layer implements earlier concepts in practice and how the proposed method integrates into an end-to-end stack. It also clarifies the runtime dataflow we evaluate later.

4.1 Overview

Our networking system is designed as an independent layer (the "network layer") positioned beneath the game system (the "game layer"). The goal is to expose minimal complexity, making game development akin to creating a single-player game. The game system comprises a "world" (representing the game state), game logic that modifies this world, and events. These events facilitate communication between different logic elements and trigger user-facing indicators, such as animations.

This separation of concerns within the game system itself facilitates a cleaner interface with the underlying networking layer, as illustrated below.

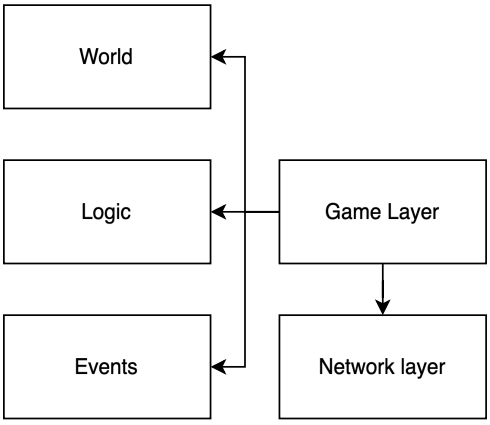

Figure 4.1: The fundamental division between the game layer, responsible for gameplay mechanics and presentation, and the network layer, responsible for communication.

4.1.1 Four quadrants model

Visually, the game and networking layers can be divided into four quadrants. The horizontal axis differentiates between the server and the client. As in Section 3, we use the classic server/client model, where the server has authority over the game world. Clients can only interact with the game world by sending their inputs, where the server will send updates to the game world (called replication) and events (for example, to trigger animations). The version of the world maintained on the client, which is partially updated through replication, is referred to as the "world view."

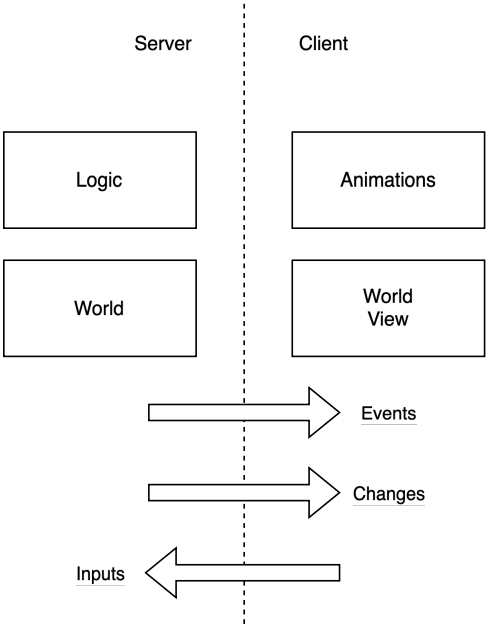

Figure 4.2: Building on this client-server distinction, we now incorporate the differentiation between the game and network layers.

4.1.2 Server and Client

The vertical axis, as previously introduced, differentiates between the game and network layers. Both the game and networking layers must serve two distinct scenarios: operating as a server or as a client. First, looking at the game layer, the server usually does not want to render the world. This independence not only boosts performance by avoiding unnecessary rendering overhead on the server, but also simplifies porting the server logic to environments without graphical capabilities.

Most game logic is also only needed on the server, as changes are replicated authoritatively to the client. An exception could be user-controlled elements of the game world, such as the player character. However, similar to the approach in Section 3, we distinctly define this as "prediction logic" to improve cohesion. Additional event-based visuals, like animations, are typically only required on the client, as they do not impact the core game logic. This ensures that client-side presentation details do not impose constraints or unnecessary complexity on the authoritative server-side game simulation.

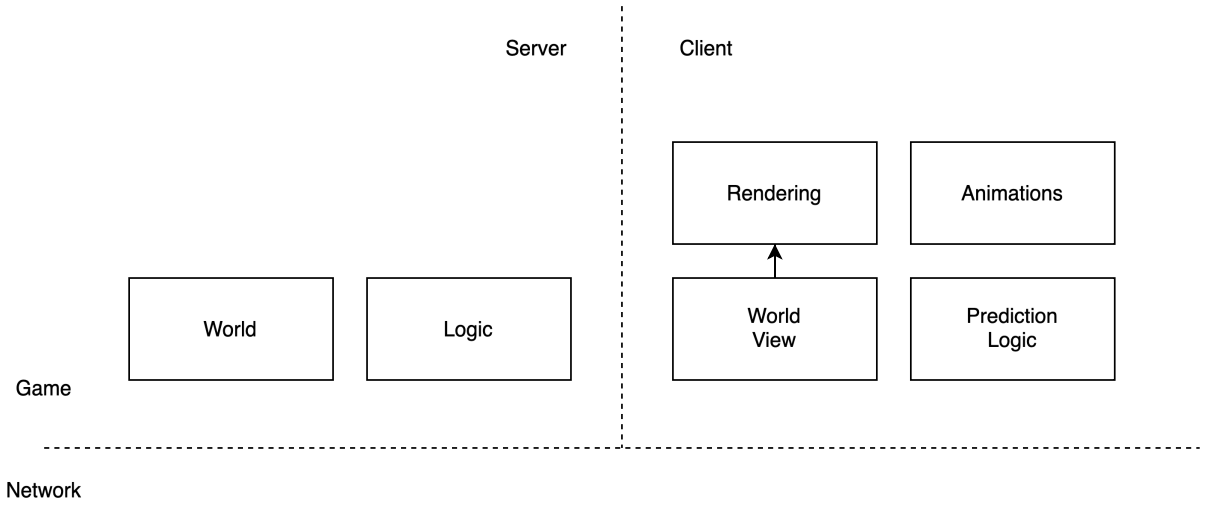

Figure 4.3: The server manages the authoritative World and Logic, while the client maintains a World View, handles Prediction Logic, Rendering, and event-based Animations.

4.1.3 Game and Network

The networking layer must cater to the server logic by providing a list of player inputs each game tick. Before replication, the networking layer has to look for changes in the server world. As introduced with "delta state" in Section 3, the system aims to send only the changes in the game world (deltas) rather than the entire world state each time. The efficiency of this delta state transmission is crucial for minimizing bandwidth usage and server load.

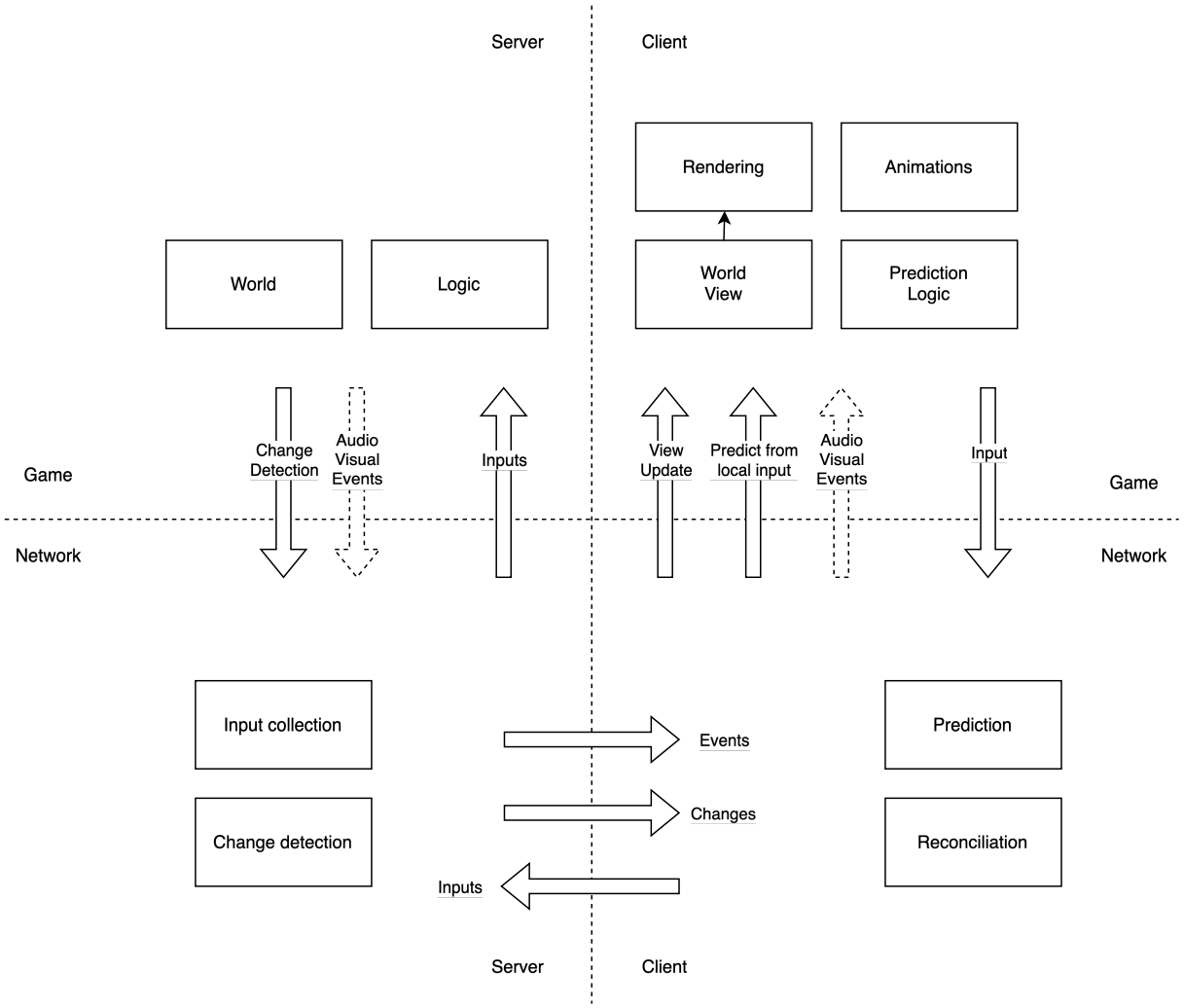

Figure 4.4: The complete four-quadrant architecture, illustrating the interplay between Game and Network layers on both Server and Client, including data flows.

Depending on how the world in the server is structured, it might provide additional helpers to detect changes. When the client receives changes, it applies them to its local game world, provided these changes are not already predicted. Otherwise, we need to use reconciliation to update our world view. This continuous loop of input collection by the network layer, server processing, state replication, and client-side reconciliation (if necessary) forms the core of the real-time networked experience.

4.2 Server networking layer

Previously, we explained an overview of our architecture and how components abstractly interact. Here, we want to dive deeper into the server-side dataflow. The following diagram displays all relevant components of the server-side networking layer, including their interaction with the game state.

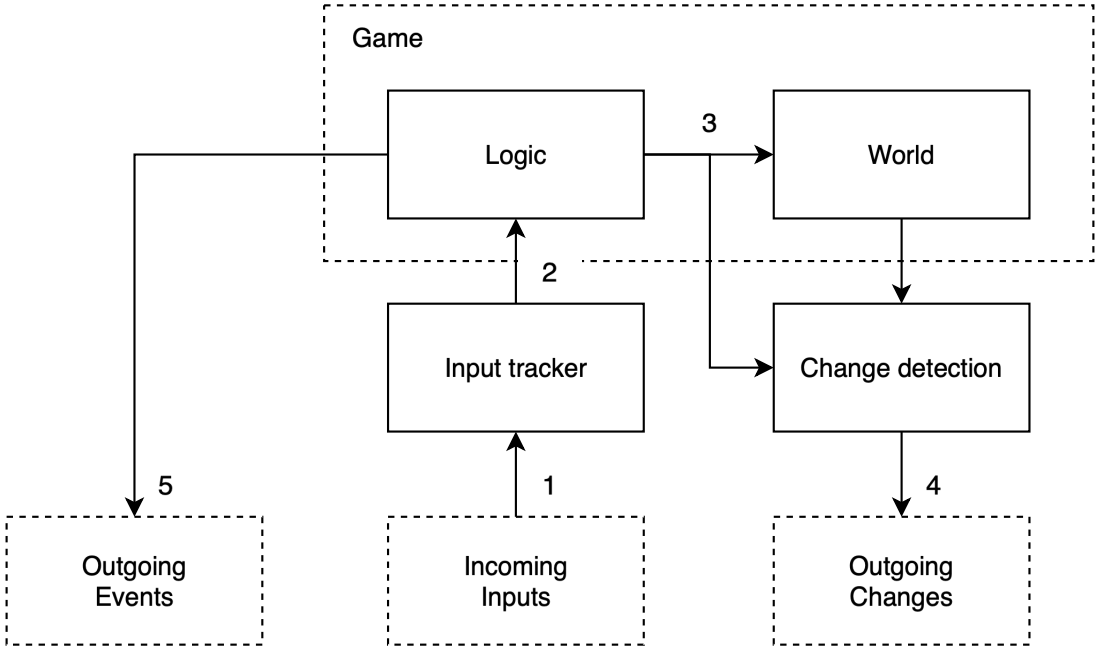

Figure 4.5: Server-side networking components and dataflow, illustrating the processing of incoming inputs and the generation of outgoing changes and events.

We have three primary communication channels with clients: incoming inputs from players, outgoing changes to replicate the game world, and outgoing events. Events are not strictly part of the game world but are needed for richer visual experiences on the client. The input tracker collects incoming inputs from players, providing a consolidated input to the game logic each tick. It is also responsible for providing stale or empty inputs if a new input has not arrived in time. The change detection mechanism is responsible for replicating the game world to all clients, ensuring each client has the same (or a consistent partial) world view by exchanging deltas.

The dataflow can be summarized by the following steps, which correspond to the numbered arrows in the diagram:

1. Users send their inputs to the server. The server receives these inputs, and the input tracker keeps track of them. Since inputs can be sent via unreliable communication methods, this component might also include logic for filtering inputs (e.g., processing only the most recent ones) or handling out-of-order packets.

2. Each tick, a set of all relevant inputs (one per player, potentially synthesized by the input tracker if an actual input is missing) is provided to the game logic. The game logic itself is fully defined in the game layer but is driven by inputs supplied by the networking layer.

3. Using these inputs, the game logic in the game layer modifies the world state. Optionally, the game logic may directly inform the change detection component about specific changes that have occurred.

4. Modifications to the world state need to be communicated to all clients. This is managed by the change detection component, which generates deltas representing these modifications. Since world state changes happen frequently and quickly, unreliable communication methods might be preferred for sending these deltas to clients. However, using unreliable transport for state

updates adds complexity, such as the need for the server (or client) to track applied changes and potentially re-send missed updates to ensure eventual consistency.

5. Processing the game logic can also trigger occurrences that are not direct modifications of the persistent game world but are important for the player experience (e.g., sound effects, temporary visual effects). These are transmitted directly to the corresponding clients as events. Events are often sent reliably if they are critical, or unreliably if they are transient and missing one isn't detrimental.

This entire process is repeated each game tick while the game is running.

4.3 Client networking layer

The overall structure and communication pathways of the client-side networking layer were introduced in the architectural overview. Having inspected the dataflow within the server-side networking layer, we now turn our attention to the client-side components. This part is particularly important for our discussion, as it is where our new algebraic reconciliation method integrates. The following diagram presents the client-side dataflow.

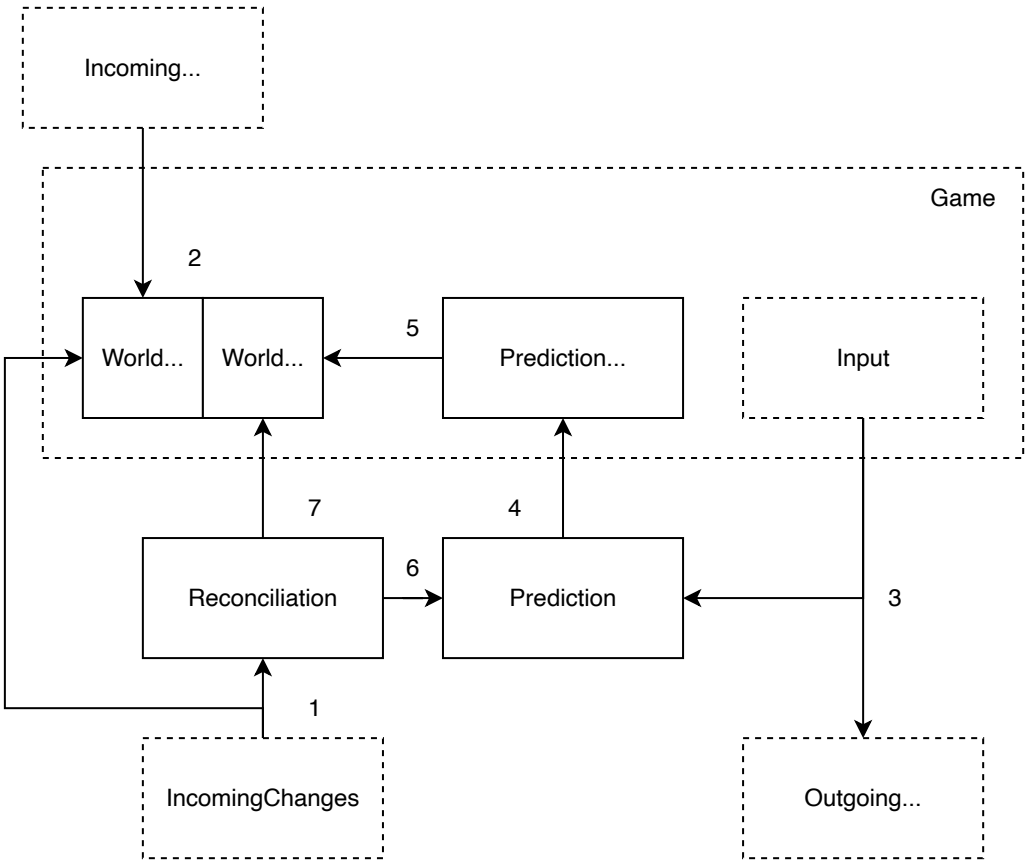

Figure 4.6: Client-side networking components and dataflow, detailing the processing of local player inputs, incoming server changes and events, and the interaction between prediction and reconciliation modules.

Similar to the server-side, the client has three primary communication channels: incoming events from the server, incoming state changes from the server, and outgoing player input. We will not focus extensively on event handling in this section, assuming that the world view component processes them (e.g., by triggering local visual or audio effects). The reconciliation, prediction, and prediction logic components collectively form the most interesting part of this architecture for our purposes. The interactions between these three elements differ depending on the specific reconciliation strategy employed, as will be detailed in the following subsections. We assume the client's representation of the game world is conceptually split into a "normal" part (the world view, reflecting acknowledged server state) and a predicted part (reflecting speculative local changes).

The dataflow can be summarized by the following steps, which correspond to the numbered arrows in the diagram:

1. Each tick, the server replicates state changes (deltas) to the client. These changes can correspond to either the predicted or non-predicted parts of the client's world representation. If a change affects a non-predicted part, it is directly applied to the world view. Otherwise, if it affects a predicted part, it is passed to the reconciliation module.
2. In parallel with receiving world state changes, the client also receives events from the server. As mentioned, we assume the world view handles these, for instance, by initiating animations or sound effects that do not alter the core game state logic.
3. Each tick, the game layer captures inputs from the player. These inputs are sent to the server as quickly as possible, often using an unreliable communication channel to minimize latency. Critically for client-side responsiveness, these inputs are also passed to the local prediction module.
4. After the prediction module receives inputs from the game layer, these inputs are fed into the game-specific prediction logic. If rollback reconciliation is being used (as detailed in Section 4.3.1), this prediction module is also responsible for memorizing these inputs for potential future re-simulation.
5. The game-layer defined prediction logic uses the local inputs to modify the predicted part of the world, most commonly affecting entities directly controlled by the player, such as their character. This immediate local modification provides the player with responsive feedback to their actions.
6. When incoming state changes from the server (received in step 1) affect parts of the world that have been locally predicted, these discrepancies must be handled by the reconciliation module. This module receives the authoritative server change and information about the current predicted state from the prediction module.
7. If rollback reconciliation is employed, the reconciliation module will use the memorized inputs (from step 4) and the authoritative server state to re-simulate the player's actions and compute a corrected predicted part of the world. For other reconciliation methods, like algebraic reconciliation (detailed in Section 4.3.2), this step will differ. After processing, the reconciliation module updates the predicted part of the world to align it more closely with the authoritative server state, aiming to correct any mispredictions smoothly.

Similarly to the server-side dataflow, this entire process is repeated each game tick while the game is running.

4.3.1 Rollback reconciliation

When an incoming authoritative state change (delta) from the server affects the predicted part of the client's game world, reconciliation is necessary to correct any mispredictions. The most common reconciliation method currently used in games is rollback reconciliation, which was introduced conceptually in Section 3.1.6. The core idea behind rollback reconciliation is to revert the client's game state to the last known authoritative state before the misprediction occurred, apply the incoming server correction, and then re-simulate all local player inputs that have occurred since that authoritative state.

The interactions for this process within our client-side architecture are illustrated below.

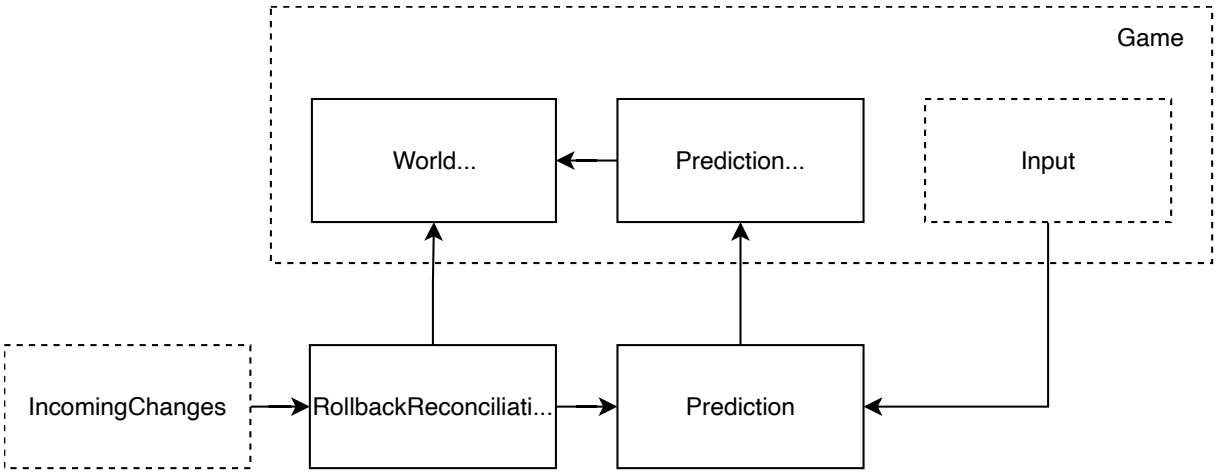

Figure 4.7: Dataflow for rollback reconciliation on the client. Incoming changes trigger a rollback, replay of memorized inputs via the prediction module, and update of the predicted world.

The primary challenge in implementing rollback reconciliation lies in efficiently reverting the game state. When working with delta states (as discussed in Section 3.1.7), two common approaches are:

1. **State Cloning**: Maintain at least two distinct copies (or snapshots) of the predicted part of the world. One copy represents the state before the latest batch of local predictions is applied. When an incoming server change necessitates a rollback, the current predicted world (with the mispredictions) is discarded, and the client reverts to the previous clean snapshot. The server change is applied to this snapshot, which then becomes the new baseline for re-applying subsequent local inputs.
2. **Storing Reverse Deltas**: For each local prediction applied to the predicted part of the world, store a corresponding "reverse delta" or "undo operation." To roll back, these reverse deltas are applied in reverse chronological order to the current predicted state, effectively reverting it step-by-step to the desired past authoritative state. Once rolled back, the server's delta is applied, and then local inputs are re-simulated.

After successfully rolling back the state to the point of divergence, the reconciliation module needs to re-apply the local inputs that the player has generated since that point. To facilitate this, the

prediction module, as mentioned in the client-side dataflow (see step 4), memorizes these applied inputs in a buffer. The rollback reconciliation process accesses this buffer to retrieve the sequence of inputs that must be re-played by the prediction logic against the corrected state.

4.3.2 Algebraic reconciliation

A significant drawback of rollback reconciliation, as highlighted in Section 3.1.6, is the computational cost of re-simulating predictions. Each time an authoritative server update corrects the client's predicted state, the client might need to re-process multiple ticks of input. This re-simulation effort can increase with the client's network latency, potentially affecting performance, especially in games with many frequently predicted game objects or for players with higher ping times.

The dataflow for applying algebraic reconciliation within our client architecture is depicted in the figure below.

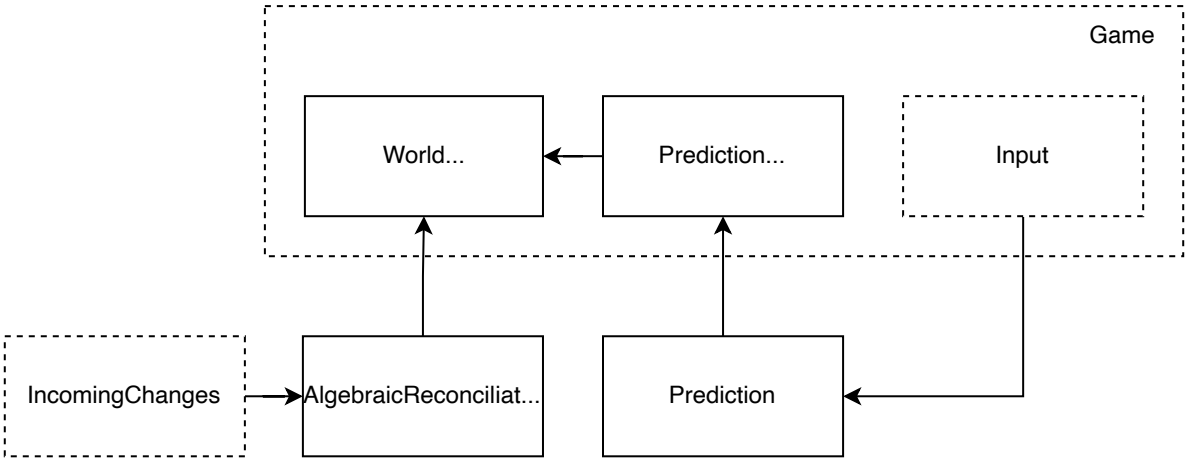

Figure 4.8: Dataflow for algebraic reconciliation on the client. Incoming server deltas are used by the reconciliation module to calculate a direct correction delta.

Our proposed algebraic server reconciliation method, whose theoretical basis was introduced in Section 3.2, offers an alternative designed to avoid this repetitive prediction overhead. This method relies on representing game state changes as "delta states." For algebraic reconciliation to work effectively, these delta states need to possess certain well-defined mathematical properties, specifically those of an abelian group, as detailed in our theory section. In essence, this means the deltas can be reliably combined, effectively undone, and the sequence of their combination does not alter the final result, much like arithmetic operations on numbers. When these conditions are met, corrections to the client's predicted state can often be calculated and applied as a single, direct adjustment, rather than requiring a full re-simulation of past inputs.

The implementation of algebraic reconciliation within the client-side networking layer primarily modifies how the reconciliation module processes incoming server changes and updates the client's predicted part of the world. The client's prediction logic, when handling local player inputs, conceptualizes its effects as a series of "predicted deltas." The prediction module tracks

the cumulative outcome of these locally generated deltas, effectively maintaining the client's speculative view of the game state built upon the last acknowledged server state.

Concurrently, the client receives authoritative state updates from the server, also in the form of deltas. When such an authoritative server delta arrives, the reconciliation module's task is to compute a "correction delta." This is achieved by comparing the server's authoritative delta for a given time period with the net predicted delta that the client had generated for that same period. The discrepancy between these two deltas isolates the misprediction. This "difference," which is itself a delta, forms the required correction. The ability to reliably calculate this correction delta hinges on the algebraic (abelian group) properties of the delta states, which allow for a meaningful "subtraction" or "inversion" of the predicted deltas relative to the server's authoritative ones, as outlined in Section 3.2.

Once this single correction delta is determined, it is applied directly to the client's current predicted part of the world. This direct application adjusts the client's state to more accurately reflect the server's view, effectively integrating the server's information without the need to rewind and individually re-process past local inputs. This avoidance of extensive input re-simulation is a key advantage over the rollback method. While the re-simulation of old inputs is bypassed, the client naturally continues to apply new local inputs via the prediction logic to its newly corrected state to maintain immediate responsiveness. It is important to note that for this process to function, the prediction module must maintain a record of the predicted deltas it has generated, as this information is essential for calculating the correction delta. This is distinct from rollback reconciliation's requirement to store the raw inputs themselves for potential re-simulation.

By directly adjusting the predicted state through these well-behaved delta manipulations, the algebraic approach can offer a more computationally efficient reconciliation process. This is particularly beneficial when the game state can be effectively represented by deltas that adhere to the required algebraic properties, as this allows for the direct calculation and application of corrections.

5 Experiments

The theoretical foundations and architectural designs presented in previous chapters establish algebraic server reconciliation as a promising alternative to traditional rollback-based methods. This chapter transitions from theory to empirical validation, demonstrating through controlled experiments that our proposed method achieves its intended goals: maintaining synchronization quality comparable to rollback reconciliation while eliminating the computational burden of repeated state re-simulation. We present a series of experiments designed to isolate and examine specific aspects of reconciliation behavior, beginning with fundamental movement scenarios and progressing to increasingly complex interactions that challenge our temporal stability assumptions.

5.1 Methodology

Our experimental approach prioritizes observable behavioral characteristics over raw performance metrics, as the computational advantages of algebraic reconciliation are mathematically self-evident from the complexity analysis presented in Chapter 3. The critical question for practical adoption is not whether algebraic reconciliation reduces computational load, it demonstrably transforms $O(n_{\text{predict}})$ operations into $O(1)$, but rather whether this efficiency comes at the cost of synchronization quality or player experience.

To enable rigorous comparison, we implemented three reconciliation strategies within a unified networking framework: rollback reconciliation representing the current industry standard, override reconciliation serving as a naive baseline, and our proposed algebraic reconciliation. Each strategy operates as an interchangeable module, ensuring that observed differences arise solely from the reconciliation method rather than implementation artifacts. The framework employs simulated network sockets with configurable round-trip delays, eliminating external network variability while maintaining realistic latency characteristics. This controlled environment enables reproducible experiments that isolate the specific behaviors we seek to understand.

The test scenarios utilize two custom-built games: a top-down movement game where players control circles in a 2D plane, and a side-scrolling game with gravity-based physics. Both games employ the Matter.js [2] physics engine to ensure realistic collision dynamics that create the state interdependencies which challenge reconciliation methods. Throughout each experiment, we track three primary metrics: state divergence quantifying the overall difference between client and server states, specific observed values such as player positions tracked from multiple perspectives, and comparison metrics measuring the absolute difference between client-predicted and server-authoritative values. All experiments operate at 60 Hz, matching common game update frequencies, with network delays configured to represent typical internet latency conditions.

5.2 Experiment 1: Isolated Movement Patterns

This initial experiment examines reconciliation behavior in a carefully controlled scenario designed to validate our fundamental theoretical predictions. By isolating player movement from complex interactions, we can observe how each reconciliation method handles the most common source of prediction errors in networked games: incomplete information about other players' actions. The experiment specifically tests whether algebraic reconciliation can match the synchronization quality of rollback reconciliation when the temporal stability assumption holds perfectly, that is, when predicted deltas remain consistent regardless of minor variations in the base state from which predictions originate.

5.2.1 Setup

The experiment employs our top-down movement game with two distinct player roles that create complementary observation perspectives. Player 1 executes a deterministic movement pattern designed to generate regular, predictable reconciliation events: 30 ticks of downward movement followed by 30 ticks of rightward movement, repeating throughout the 120-tick (2-second) experiment duration. This pattern ensures that velocity changes occur at known intervals, creating periodic challenges for the reconciliation system as the client must correct its predictions when these changes become known. Player 2 remains stationary at a fixed position, serving as an observer whose client must reconcile Player 1′s movements without any local prediction of its own movement.

The network configuration simulates moderate latency conditions with a 30-tick delay in each direction, representing approximately 1000ms round-trip time at 60 Hz, typical of intercontinental internet connections. This delay is substantial enough to create meaningful prediction windows where clients must operate on incomplete information, yet not so extreme as to represent unrealistic edge cases. The absence of collision interactions between players ensures that prediction errors arise solely from information delay rather than complex physical dependencies, allowing us to evaluate the pure reconciliation mechanics of each method.

5.2.2 Results

The experimental results reveal striking similarities between algebraic and rollback reconciliation, validating our theoretical predictions about convergence behavior when temporal stability assumptions are satisfied. We present three perspectives that collectively demonstrate the effectiveness of algebraic reconciliation in maintaining both state consistency and prediction responsiveness.

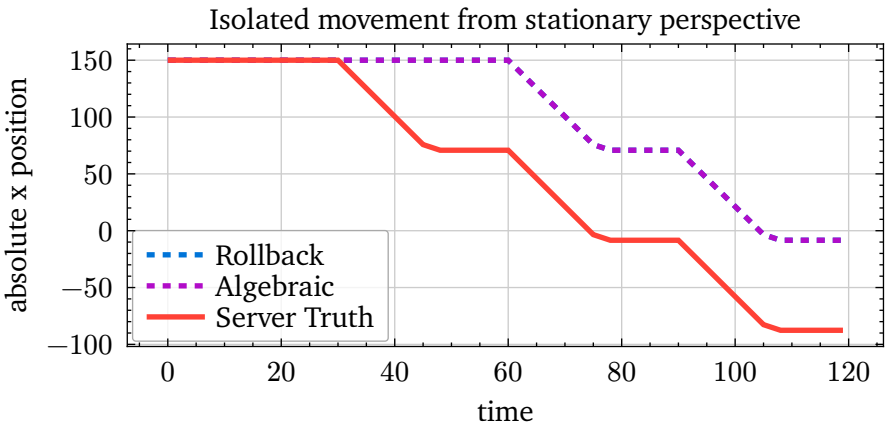

Figure 5.1: Player 1's x-position as observed from Player 2's client perspective. The server's authoritative position (solid red) is compared with the client's synchronized view under algebraic (purple dashed), rollback (blue dashed), and override (red dashed) reconciliation strategies. The complete overlap of all three client lines demonstrates perfect convergence across all methods.

Figure 5.1 presents the most fundamental test of reconciliation correctness: the ability to maintain consistent state for entities not under local control. From Player 2′s stationary perspective, all three reconciliation methods successfully track Player 1′s movement pattern, with the dashed lines representing client state perfectly overlapping across algebraic, rollback, and override strategies. This overlap is particularly significant for algebraic reconciliation, as it demonstrates that the correction term $\varepsilon = \Delta s - \Delta p$ accurately compensates for unpredicted movements without requiring state rollback or input re-simulation. The periodic shifts in the observed position correspond to Player 1′s velocity changes, which propagate to Player 2′s client after the network delay. The fact that algebraic reconciliation handles these discontinuous changes as effectively as rollback reconciliation validates our approach to direct algebraic correction.

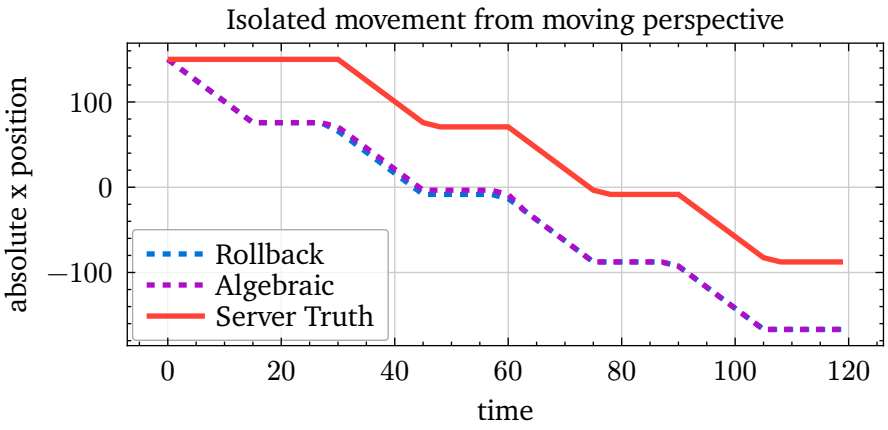

Figure 5.2: Client-side prediction accuracy from Player 1's perspective. Both algebraic (purple dashed) and rollback (blue dashed) reconciliation produce identical predictions that maintain a consistent offset from the server's authoritative state (red solid), demonstrating equivalent prediction quality.

The perspective from Player 1′s client, shown in Figure 5.2, reveals the prediction dynamics that make modern networked games playable despite latency. Both algebraic and rollback reconciliation maintain identical prediction trajectories, consistently leading the server's authoritative position by approximately 30 ticks, exactly the network delay period. This offset represents the fundamental characteristic of client-side prediction: the client shows where the player will be once the server processes their inputs, not where the server currently believes them to be. The perfect

alignment between algebraic and rollback predictions confirms that our method preserves the essential responsiveness of client-side prediction while eliminating the computational overhead of repeated simulation. The smooth transitions at velocity change points (every 30 ticks) demonstrate that both methods handle prediction updates gracefully, without introducing additional artifacts or discontinuities.

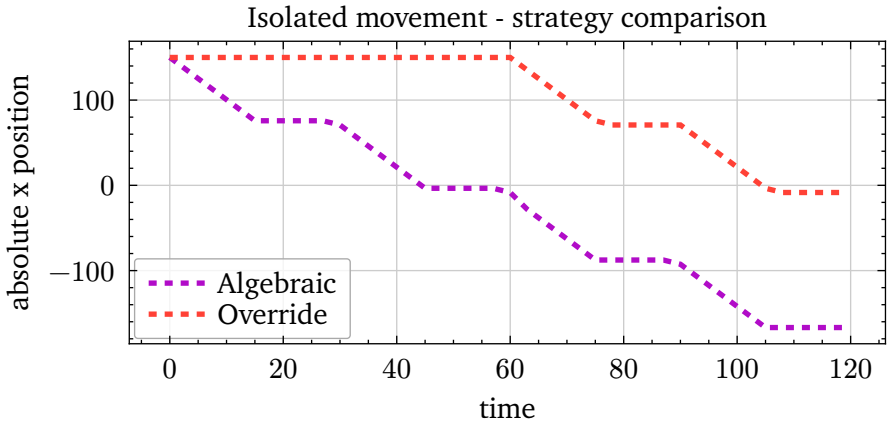

Figure 5.3: Comparison of prediction responsiveness between algebraic (purple dashed) and override (red dashed) reconciliation from Player 1's perspective. The stair-step pattern in override reconciliation reflects the 30-tick delay between input and visible effect, while algebraic reconciliation provides immediate visual feedback.

Figure 5.3 starkly illustrates why naive override reconciliation creates an unacceptable player experience in interactive games. The override method's characteristic stair-step pattern reveals the full network round-trip delay between player input and visual feedback: each movement command initiated by Player 1 only becomes visible after traveling to the server and back, creating a 60-tick (1-second) perceived lag. In contrast, algebraic reconciliation maintains smooth, immediate response to inputs, with position updates appearing instantaneously from the controlling player's perspective. The periodic convergence points where both lines intersect represent moments when the server state finally catches up to where the client was 30 ticks earlier, confirming that both methods eventually achieve consistency despite their radically different user experiences. This comparison underscores that algebraic reconciliation preserves the essential benefit of client-side prediction, responsive controls, while achieving it through direct algebraic operations rather than speculative simulation.

5.2.3 Analysis

The results from this foundational experiment provide strong empirical support for algebraic server reconciliation as a viable alternative to rollback-based methods. Several key observations emerge from the data that validate our theoretical framework and demonstrate practical applicability.

The complete convergence achieved by all three methods when observing non-controlled entities confirms that algebraic reconciliation correctly handles the most common case in multiplayer games: synchronizing the positions of other players. The mathematical elegance of the correction formula $\varepsilon = \Delta s - \Delta p$ translates directly into practical effectiveness, producing state updates indistinguishable from those achieved through the more computationally expensive rollback and re-simulation process. This equivalence holds even across discontinuous velocity changes, demonstrating robustness to the types of sudden state transitions common in interactive games.

From the controlling player's perspective, algebraic reconciliation achieves the critical goal of preserving prediction quality while eliminating re-simulation overhead. The identical prediction trajectories produced by algebraic and rollback methods confirm that our approach maintains the same subjective experience of responsiveness that makes client-side prediction essential for networked games. With a 30-tick network delay, rollback reconciliation would need to re-simulate up to 30 frames of game logic for each server update, a computational burden that algebraic reconciliation reduces to a single vector addition operation.

The stark contrast with override reconciliation reinforces the necessity of prediction-based approaches for interactive games. The one-second perceived lag demonstrated by override reconciliation would render most action games unplayable, yet this is precisely the experience players would have without some form of client-side prediction. Algebraic reconciliation achieves the same responsive feel as rollback methods while requiring significantly less computational resources, making it particularly attractive for mobile platforms or games with numerous predicted entities.

This experiment establishes that under conditions where the temporal stability assumption holds, such as simple movement without complex state interactions, algebraic server reconciliation performs identically to rollback reconciliation from a behavioral perspective while offering clear computational advantages. The next experiments will explore how this performance translates to scenarios involving physical collisions and state-dependent dynamics that challenge our foundational assumptions.

5.3 Experiment 2: Collision Dynamics and State Dependencies

The previous experiment validated algebraic reconciliation under ideal conditions where state changes remain independent and predictable. Real-world game scenarios, however, frequently involve complex interactions between entities that create state dependencies and violate our temporal stability assumptions. This experiment introduces physical collisions between players to examine how algebraic reconciliation behaves when physics constraints force states to diverge from their predicted trajectories. By analyzing the reconciliation behavior during and after collision events, we can identify the practical limitations of our method and understand the error characteristics that emerge when mathematical assumptions encounter physical reality.

5.3.1 Setup

This experiment employs the same top-down movement game but introduces a direct collision scenario designed to maximally stress the reconciliation system. Two players are positioned to create an inevitable head-on collision: Player 1 starts at position (-200, 150) moving rightward at constant velocity, while Player 2 begins at (200, 140) moving leftward at the same speed. The slight vertical offset ensures the collision occurs at an angle, creating forces in both x and y dimensions that challenge the prediction system in multiple axes simultaneously.

The physics engine enforces a hard constraint that player collision volumes cannot overlap, forcing the simulation to resolve conflicts by pushing players apart when they attempt to occupy the same space. This constraint fundamentally violates our temporal stability assumption: when a client predicts its own movement without knowledge of the impending collision, it generates deltas that assume unimpeded motion. The server, processing both players' movements simultaneously, generates different deltas that account for the collision forces. This divergence between predicted

and actual deltas provides an ideal test case for understanding how algebraic reconciliation handles state-dependent interactions.

We conduct the experiment at three different network delays, 15, 30, and 45 frames, to observe how latency affects the magnitude and duration of reconciliation errors. Each test runs for 120 frames (2 seconds at 60 Hz), sufficient to capture the complete collision event including approach, contact, separation, and post-collision reconciliation. The collision typically occurs around frame 60, allowing observation of both pre-collision prediction accuracy and post-collision error recovery.

5.3.2 Results

The experimental data reveals distinct behavioral patterns in how algebraic reconciliation handles collision-induced state dependencies, with notable differences between axis behavior and observation perspective. We present the results organized by axis to highlight the different error characteristics that emerge in the primary movement direction versus the perpendicular collision response direction.

X-Axis Behavior

The x-axis represents the primary movement direction for both players, where intentional inputs drive the motion and collisions directly oppose the intended movement. This axis best demonstrates how reconciliation methods handle the conflict between predicted motion and physics-constrained reality.

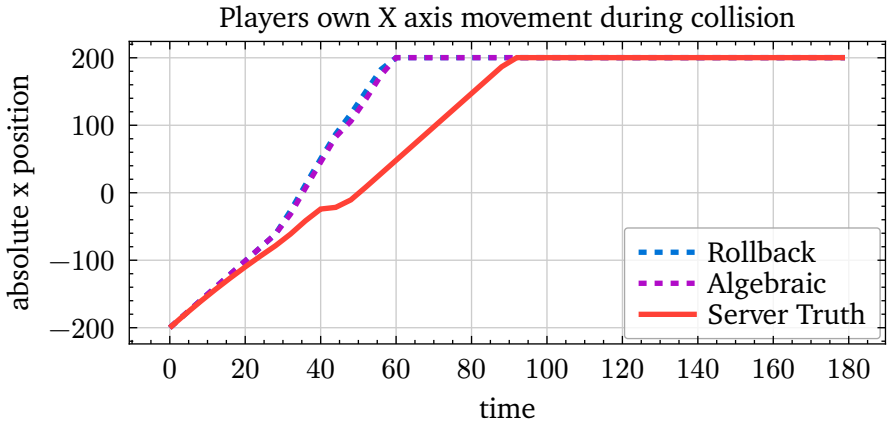

Figure 5.4: Player 1's x-position observed from their own perspective during collision. Both algebraic (purple dashed) and rollback (blue dashed) reconciliation show similar overshooting behavior relative to the server state (red solid), indicating comparable prediction behavior when movement intentions conflict with physics constraints.

Figure 5.4 demonstrates that from the controlling player's perspective, algebraic and rollback reconciliation produce nearly identical predictions along the primary movement axis. Both methods overshoot the server's position by approximately 40 units at the peak, reflecting the fundamental challenge of predicting through an unknown collision. The overshoot occurs because the client continues predicting rightward movement while the server has already processed the collision and stopped forward progress. The subsequent convergence shows both methods successfully reconciling to the server state once the collision information propagates through the network delay. The slight divergence between algebraic and rollback traces during the collision phase

(frames 50-90) suggests minor differences in how the methods handle the rapid state changes, but these differences remain visually negligible from the controlling player's perspective.

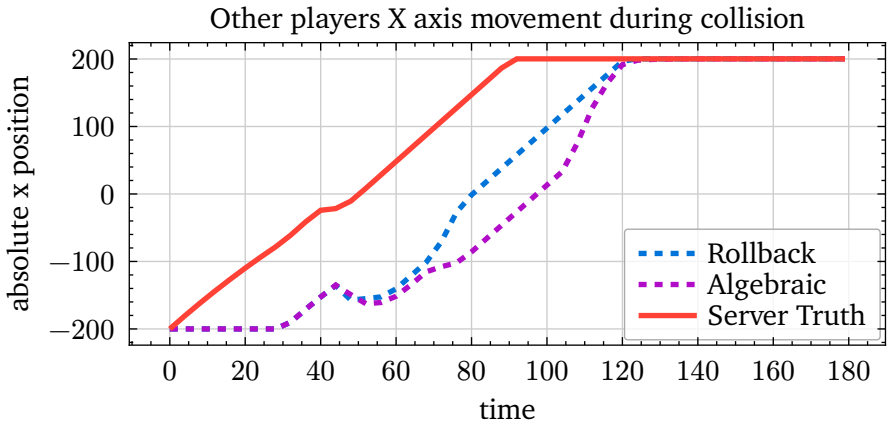

Figure 5.5: Player 1's x-position as observed from Player 2's perspective. The algebraic method (purple dashed) shows slightly greater deviation from the server state compared to rollback (blue dashed), revealing increased error when observing other players during collisions.

The view from Player 2's perspective, shown in Figure 5.5, reveals a subtle but important difference between the reconciliation methods. The algebraic approach produces slightly larger deviations from the server state when observing the other player's collision response. This increased error for non-controlled entities suggests that the algebraic correction term $\varepsilon = \Delta s - \Delta p$ becomes less accurate when applied to states affected by multi-entity interactions, as the correction computed for one player's movement doesn't fully account for the coupled dynamics introduced by the collision.

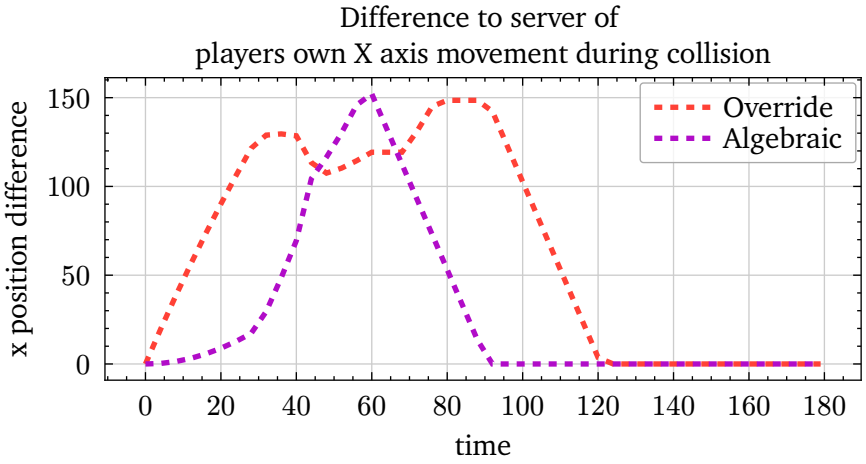

Figure 5.6: Absolute difference from server x-position for Player 1's own view. Algebraic reconciliation (purple dashed) maintains significantly lower error than override reconciliation (red dashed), though both methods struggle during the collision event.

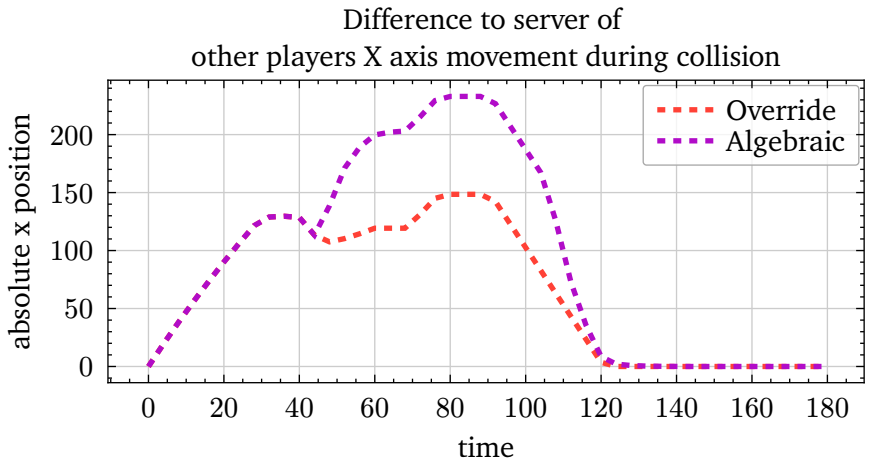

Figure 5.7: Absolute difference from server x-position when Player 2 observes Player 1. The algebraic method (purple dashed) shows periods of higher error than override (red dashed), particularly during collision resolution.

Figures Figure 5.6 and Figure 5.7 compare the absolute error magnitudes between algebraic and override reconciliation. From the controlling player's perspective, algebraic reconciliation achieves substantially lower error throughout most of the simulation, with peak errors during collision approximately 50% lower than override. However, when observing other players, algebraic reconciliation occasionally produces higher errors than override, particularly during the collision resolution phase. This asymmetry suggests that while algebraic reconciliation excels at maintaining responsive control for the local player, it may introduce artifacts when synchronizing observed entities during complex interactions.

Y-Axis Behavior

The y-axis behavior provides unique insights into reconciliation accuracy because neither player intentionally moves along this axis, all y-displacement results from collision forces. This allows us to isolate the reconciliation of unpredicted, physics-induced state changes from the reconciliation of intended movements.

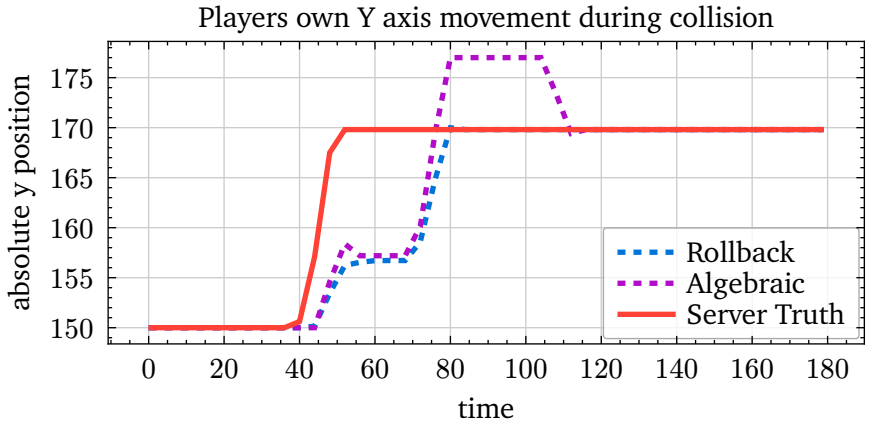

Figure 5.8: Player 1's y-position during collision as seen from their own perspective. The algebraic method (purple dashed) exhibits characteristic overcorrection, overshooting the server position (red solid) more dramatically than rollback reconciliation (blue dashed).

Figure 5.8 reveals the most distinctive characteristic of algebraic reconciliation under violated assumptions: systematic overcorrection. When the server's collision-induced y-displacement

arrives at the client, the algebraic method applies this correction to a state that has already been predicting collision effects based on incomplete information. The result is a compounding of corrections that pushes the y-position beyond the server's value by approximately 20 units. This overcorrection gradually resolves as subsequent server updates arrive, but the pattern clearly shows the algebraic method struggling to handle state changes that depend on information not available during prediction.

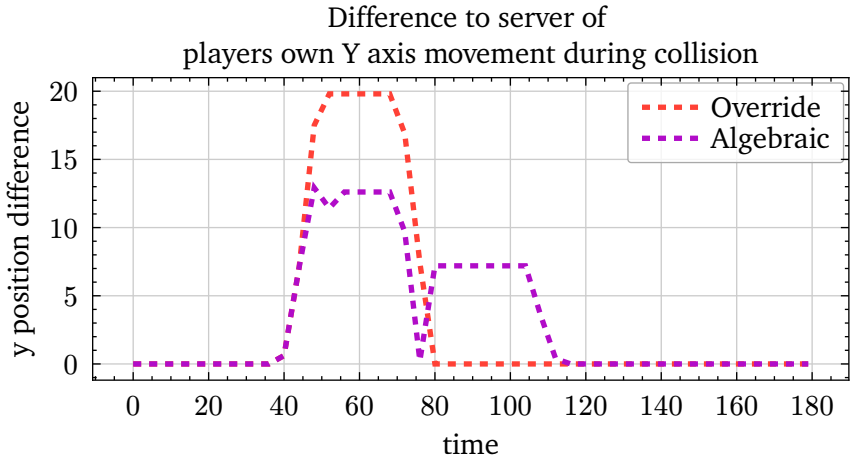

Figure 5.9: Absolute y-position error from Player 1's perspective. The algebraic method (purple dashed) shows a characteristic double-peak pattern, with the second peak representing the overcorrection artifact unique to this reconciliation approach.

The error analysis in Figure 5.9 provides crucial insight into the overcorrection mechanism. Both methods exhibit an initial error peak when the collision occurs without prediction, but algebraic reconciliation uniquely produces a second, smaller peak approximately 30 frames later. This secondary peak corresponds to the overcorrection being resolved as the server state confirms the players have separated. The double-peak pattern is pathognomonic of algebraic reconciliation's response to state-dependent dynamics: the method first overcorrects when applying delayed corrections to an already-evolved state, then must correct the overcorrection once the dependency resolves.

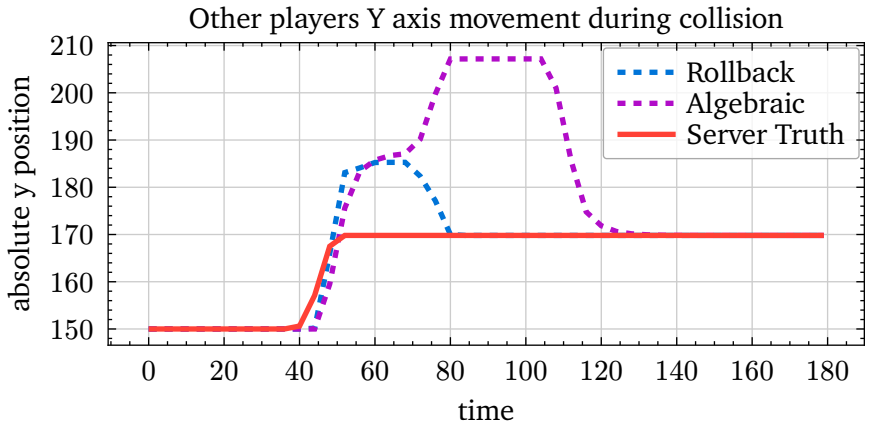

Figure 5.10: Player 1's y-position observed from Player 2's perspective. The algebraic method (purple dashed) shows substantially larger deviation from the server state (red solid) compared to rollback (blue dashed).

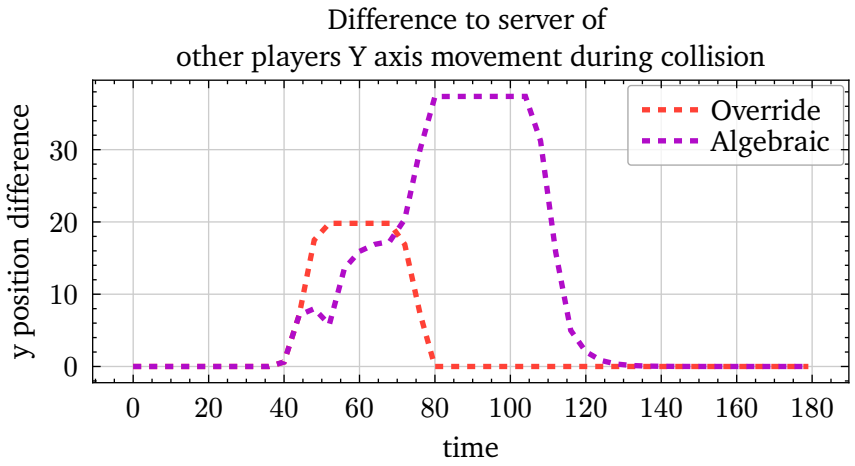

Figure 5.11: Y-position error magnitude when Player 2 observes Player 1. The algebraic method (purple dashed) produces errors more than twice as large as rollback reconciliation (blue dashed), highlighting the method's weakness in synchronizing observed collision dynamics.

The degradation of algebraic reconciliation becomes most apparent when examining y-axis synchronization from the observing player's perspective, shown in Figures Figure 5.10 and Figure 5.11. The error magnitude for algebraic reconciliation exceeds rollback by a factor of two or more during the collision event, with peak errors reaching nearly 50 units compared to rollback's 20-unit maximum. This dramatic difference indicates that algebraic reconciliation's assumptions break down severely when reconciling observed entities whose states depend on complex multi-body interactions.

5.3.3 Analysis

The collision experiment exposes both the capabilities and limitations of algebraic server reconciliation when faced with state-dependent dynamics that violate temporal stability assumptions. Several critical insights emerge from the data that inform practical deployment decisions.

The method demonstrates acceptable performance along axes where player input directly drives motion, maintaining prediction quality comparable to rollback reconciliation from the controlling player's perspective. This suggests that even during complex interactions, algebraic reconciliation can preserve the responsive feel essential for playability when the primary motion remains input-driven. The systematic overcorrection observed in the perpendicular axis, while visually detectable, remains within tolerable bounds for the controlling player and resolves within approximately one network round-trip time.

However, the significant error amplification when observing other players during collisions represents a serious limitation. The two-fold or greater increase in synchronization error compared to rollback reconciliation could produce visible artifacts in multiplayer scenarios with frequent player interactions. The root cause appears to be the fundamental assumption that predicted deltas remain stable regardless of base state variations, an assumption thoroughly violated when collision forces create coupled dynamics between multiple entities.

The asymmetric error characteristics, lower error for controlled entities, higher error for observed entities, suggest a hybrid deployment strategy might be optimal. Games could employ algebraic reconciliation for player-controlled entities where responsiveness is paramount and computational savings are valuable, while using rollback reconciliation for observing other players where

accuracy matters more than computational efficiency. This selective application would leverage the strengths of each method while mitigating their respective weaknesses.

The temporal pattern of overcorrection and recovery provides guidance for potential improvements to the algebraic method. The predictable nature of the overcorrection, always occurring one network delay after the initial collision, suggests that adaptive correction factors or damping terms could reduce the artifact magnitude without sacrificing the method's computational advantages. Future work might explore such enhancements to extend algebraic reconciliation's applicability to more complex interaction scenarios.

5.4 Discussion

The exploration of algebraic server reconciliation presented in this thesis reveals a fundamental truth about networked game development: there exists no universal solution to the synchronization problem, but rather a spectrum of approaches each with distinct trade-offs and optimal use cases. Through our theoretical analysis and experimental validation, we have demonstrated that algebraic reconciliation offers a computationally efficient alternative to traditional rollback methods under specific conditions. This chapter synthesizes our findings within the broader context of game networking, examining the practical implications for game developers and the relationship between our method and existing distributed systems techniques.

5.4.1 The Landscape of Networking Optimizations

Our investigation has revealed that networking approaches in games are not monolithic solutions but rather collections of techniques that can be optimized for specific scenarios. Consider rollback reconciliation: while computationally expensive in the general case, it becomes remarkably efficient when only small portions of the game world require prediction. A real-time strategy game where players control a single hero unit among thousands of AI-controlled entities can implement rollback exclusively for the hero, achieving both responsiveness and efficiency. Similarly, games with predominantly static environments can minimize the reconciliation overhead by limiting prediction to dynamic objects.

The diversity of optimization opportunities extends beyond traditional methods. Minecraft's approach of sending predicted states from client to server, rather than just inputs, demonstrates how architectural decisions can reduce development complexity at the cost of other properties [12]. By accepting the client's predicted state and only validating it server-side, Minecraft eliminates the need for complex reconciliation logic. This design choice makes the networking layer simpler to implement and maintain, particularly valuable for smaller development teams. However, this simplicity comes with clear trade-offs in security and consistency guarantees [12].

5.4.2 Trade-offs in Network Architecture Design

The fundamental tension in networked game design exists between three competing goals: performance, consistency, and security. Our algebraic reconciliation method optimizes for performance by eliminating re-simulation, maintains reasonable consistency through mathematical convergence properties, but like all prediction-based approaches, cannot guarantee perfect security against malicious clients. This three-way trade-off manifests differently across various architectural choices.

The Minecraft approach prioritizes development simplicity and performance over security, accepting client-authoritative updates that can be exploited by modified clients. Traditional rollback reconciliation maximizes consistency and security at the cost of computational performance. Deterministic lockstep, as discussed in Chapter 2, achieves perfect consistency and security but sacrifices performance and responsiveness. Our algebraic method occupies a middle ground, offering improved performance over rollback while maintaining similar security properties, though with reduced consistency guarantees when temporal stability assumptions are violated.

These trade-offs are not merely technical considerations but fundamentally shape the player experience and viable game designs. A competitive esports title cannot compromise on security, making client-authoritative approaches unacceptable regardless of their performance benefits. Conversely, a cooperative sandbox game might reasonably prioritize creative freedom and performance over strict security enforcement.

5.4.3 Connections to Established Distributed Systems Techniques

The algebraic reconciliation method introduced in this thesis draws inspiration from and extends techniques widely deployed in distributed systems, particularly Conflict-Free Replicated Data Types (CRDTs) [9]. The connection is more than superficial: both approaches rely on algebraic properties to ensure convergence without coordination. However, our method exploits the specific topology of client-server game architectures to achieve properties that generic CRDTs cannot provide.

Traditional CRDTs must handle arbitrary ordering of operations from any replica, requiring strong commutativity properties that severely limit the types of operations that can be supported. Our algebraic reconciliation relaxes this requirement by leveraging the authoritative server model. We do not need operations to commute with all possible operations, only with the specific prediction deltas generated by clients. This relaxation enables support for a broader class of game state transformations while maintaining convergence guarantees.

Furthermore, our centralized authority model eliminates the need for complex conflict resolution mechanisms inherent in peer-to-peer CRDTs. The server's authoritative state provides a natural linearization point that resolves any conflicts through the reconciliation process. This architectural advantage allows us to handle operations that would create conflicts in a pure CRDT system, such as competitive resource allocation or exclusive state transitions.

5.4.4 Limitations and Domain Constraints

Despite its advantages, algebraic reconciliation faces fundamental limitations that restrict its applicability. The most significant constraint emerges in games with strong state dependencies, where the behavior of one entity fundamentally alters the dynamics of another. Our experiments with collision dynamics demonstrated this limitation clearly: when physics constraints couple entity movements, the temporal stability assumption fails, leading to overcorrection artifacts.

The limitation becomes insurmountable in certain game architectures, particularly voxel-based games like Minecraft or Terraria. In these games, blocks lack persistent identity beyond their position in the world grid. When a player mines a block at position (x, y, z), that block ceases to exist rather than moving or transforming. This deletion-creation pattern violates the assumptions underlying our abelian group construction, which requires entities to maintain identity across

state changes. Without stable entity identities, we cannot meaningfully compute difference deltas or apply corrections algebraically.

Similarly, games featuring complex emergent behaviors from simple rules, such as Conway's Game of Life simulations or cellular automata-based mechanics, resist algebraic reconciliation. The state at each timestep depends holistically on the entire previous state, making it impossible to isolate independent deltas that can be algebraically composed. These systemic dependencies create cascading prediction errors that our correction mechanism cannot resolve.

5.4.5 Applicability and Adoption Potential

Despite these limitations, a significant portion of the multiplayer gaming landscape could benefit from algebraic reconciliation. The method excels in games with:

- Entity-based architectures where objects maintain persistent identities
- Relatively independent entity behaviors with occasional interactions
- Position-based movement as the primary state change
- Limited physics coupling between player-controlled entities

This encompasses many popular genres including MOBAs, battle royales with large maps, MMORPGs, and real-time strategy games. Even games with occasional physics interactions, such as fighting games or sports simulations, could employ algebraic reconciliation during non-contact phases while switching to rollback during collisions.

The computational savings become particularly compelling for mobile platforms, where battery life and thermal constraints limit the feasibility of extensive re-simulation. A mobile MOBA could use algebraic reconciliation to support more players per match or reduce battery consumption without sacrificing responsiveness. Cloud gaming platforms could reduce server costs by eliminating redundant simulation work across thousands of concurrent sessions.

5.4.6 Toward Hybrid Network Architectures

Our research suggests that future networking architectures will increasingly adopt hybrid approaches, selectively applying different synchronization methods to different aspects of game state. Rather than viewing networking methods as mutually exclusive alternatives, developers should consider them as complementary tools within a comprehensive synchronization strategy.

A sophisticated implementation might partition game state into multiple synchronized domains. Player movement and simple projectiles could use algebraic reconciliation for efficiency. Complex physics interactions could employ rollback reconciliation for accuracy. Environmental changes might use event-based replication for simplicity. Inventory and progression systems could leverage CRDT-inspired structures for robustness. This compositional approach allows each subsystem to use the most appropriate synchronization method for its specific requirements.

Large-scale productions already employ similar hybrid strategies, though often in an ad-hoc manner. Our formalization provides a theoretical framework for reasoning about these compositions systematically. By understanding the mathematical properties required for each synchronization method, developers can make informed decisions about state partitioning and method selection.

5.4.7 Implications for Game Development Practice

The practical implications of our work extend beyond the specific technique of algebraic reconciliation. Our formal framework for reasoning about game state synchronization provides value independent of whether developers adopt our specific method. The explicit formulation of game state as (S, s_0, f) with progression functions and delta states offers a lingua franca for discussing and comparing networking approaches.

For small indie studios, the primary value may be in understanding the trade-offs between different approaches rather than implementing custom solutions. Knowing when rollback reconciliation becomes prohibitively expensive or when client-authoritative updates provide acceptable security can inform architectural decisions early in development when changes remain feasible.

Larger studios with resources for custom networking solutions can use our framework to identify optimization opportunities within their existing architectures. The mathematical characterization of state changes as elements of an abelian group provides a concrete test for whether algebraic reconciliation could benefit specific game systems. Even partial adoption for suitable subsystems could yield measurable performance improvements.

6 Conclusion and Future work

The theoretical foundations and experimental validation presented in this thesis establish algebraic server reconciliation as a viable technique for specific game architectures, yet this work represents only an initial exploration of a broader design space. The insights gained from our investigation reveal multiple promising research directions that could extend the applicability, efficiency, and robustness of algebraic approaches to game state synchronization. This chapter summarizes contributions and outlines the most compelling avenues for future work, ranging from immediate practical extensions to more speculative theoretical explorations.

6.1 Contributions and Broader Impact

This thesis makes three primary contributions to the field of game networking:

First, we introduce algebraic server reconciliation as a novel synchronization method that eliminates re-simulation overhead while maintaining prediction responsiveness. The technique's mathematical foundation in abelian group theory provides formal convergence guarantees under well-defined conditions.

Second, we provide a comprehensive formalization of game networking concepts that enables rigorous comparison of different approaches. This framework facilitates reasoning about hybrid architectures and helps identify which synchronization methods suit specific game mechanics.

Third, we demonstrate how to construct abelian groups for arbitrary ECS-based games, providing a practical blueprint for implementing algebraic reconciliation in modern game engines. This construction methodology extends beyond our specific technique, offering insights into how game state can be structured to support various algebraic operations.

Beyond these direct contributions, our work bridges the gap between distributed systems theory and game development practice. By adapting CRDT concepts to the specific constraints and opportunities of client-server game architectures, we demonstrate how theoretical insights from adjacent fields can yield practical benefits in game development. This cross-pollination suggests that further exploration of distributed systems techniques could yield additional innovations in game networking.

The formal treatment of game state synchronization also opens avenues for automated verification and testing of networking code. The mathematical properties we identify could form the basis for property-based testing frameworks that automatically verify synchronization correctness. Static analysis tools could detect when game mechanics violate the assumptions required for specific networking methods, preventing subtle bugs that only manifest under specific network conditions.

6.2 Hybrid Reconciliation Architectures

Perhaps the most immediate and practical extension of our work lies in the systematic development of hybrid reconciliation models. Our experiments demonstrated that algebraic reconciliation excels for position-based movement with minimal state dependencies but struggles with complex physics interactions. This suggests that optimal networking architectures should employ different reconciliation strategies for different components of game state, selecting the most appropriate method based on the specific characteristics of each state partition.

A hybrid architecture would partition game state into reconciliation domains based on their mathematical properties and interaction patterns. Entity positions and simple attributes could employ algebraic reconciliation for computational efficiency. Complex physics interactions could switch to rollback reconciliation when collisions are detected or predicted. Discrete state changes, such as ability activations or item pickups, might use event-based replication with client-side prediction but no reconciliation. Environmental modifications could leverage CRDT-inspired structures for their natural conflict resolution properties.

The key research challenge lies in managing the boundaries between these domains. When a projectile transitions from free flight (suitable for algebraic reconciliation) to collision with a character (requiring rollback), the handoff must be seamless and maintain consistency across all clients. This requires formal methods for composing different reconciliation strategies and proving that the composition maintains convergence properties. The development of such compositional frameworks would enable developers to construct sophisticated networking systems from well-understood primitive operations.

6.3 Industrial Validation Through Full Game Implementation

While our experiments demonstrate the theoretical viability of algebraic reconciliation, the ultimate test of any networking technique lies in its deployment within a production game. Future work should focus on implementing a complete multiplayer game that leverages algebraic reconciliation as its primary synchronization method. This implementation would serve multiple purposes: validating the technique under realistic conditions, identifying practical engineering challenges not apparent in controlled experiments, and demonstrating industry relevance to potential adopters.

The ideal validation project would be a game specifically designed to showcase the strengths of algebraic reconciliation while remaining genuinely entertaining. A multiplayer action-RPG with hundreds of AI-controlled enemies, where players control heroes with position-based abilities, would highlight the computational advantages when only a small fraction of entities require prediction. The game should include sufficient complexity to stress the networking system, multiple simultaneous players, varied network conditions, and diverse interaction types, while avoiding mechanics that fundamentally violate temporal stability assumptions.

Such an implementation would also reveal integration challenges with existing game engines and networking middleware. Most commercial engines assume rollback-based reconciliation in their networking layers, requiring significant architectural modifications to support algebraic methods. Documenting these integration challenges and developing migration strategies would lower adoption barriers for existing projects considering algebraic reconciliation.

6.4 Merkle Tree Optimization for Selective State Transmission

Our current implementation transmits all state changes to all clients, regardless of whether those changes affect predicted state. Future work should explore hierarchical state representations that enable selective transmission based on prediction accuracy. Inspired by Minecraft's approach to state validation, we propose using Merkle trees [8] to efficiently identify and transmit only mispredicted state components.

Under this optimization, clients would compute cryptographic hashes of their predicted state partitions and transmit these to the server alongside their inputs. The server, maintaining the authoritative state, can compare these hashes with its own to identify divergent partitions without examining the full state. Only partitions with hash mismatches require synchronization, potentially reducing bandwidth requirements by an order of magnitude for states that are accurately predicted.

The Merkle tree structure provides additional benefits beyond bandwidth optimization. The hierarchical nature enables progressive refinement, where coarse-grained divergences are corrected before fine-grained details. This could reduce the visual impact of corrections by spreading them across multiple frames. Furthermore, the cryptographic properties provide a foundation for cheat detection: clients that consistently report incorrect hashes for non-predicted state components may be running modified game code.

6.5 Higher-Order Derivative Synchronization

Our algebraic reconciliation operates on first-order derivatives (deltas) of game state. A natural theoretical extension considers whether operating on higher-order derivatives could provide additional benefits, particularly for addressing the limitations observed with acceleration-based movement and physics interactions. Just as delta states capture position changes, second-order deltas could capture velocity changes, potentially enabling accurate reconciliation of ballistic trajectories and falling objects that currently violate our temporal stability assumptions.

Consider a projectile with initial velocity v_0 and constant acceleration a due to gravity. Current algebraic reconciliation fails because position deltas change each frame as velocity increases. However, the second derivative (acceleration) remains constant, suggesting that synchronizing acceleration rather than position could maintain temporal stability even for accelerating objects. The reconciliation formula would extend to $\varepsilon = (\Delta^2 s - \Delta^2 p)$, where Δ^2 represents second-order differences.

This approach could elegantly handle several problematic scenarios identified in our experiments. Falling players would be characterized by a constant downward acceleration rather than changing position deltas. Projectiles following parabolic trajectories would require only initial velocity and acceleration vectors. Even complex spring-damper systems could be represented through their differential equations rather than explicit position updates.

The mathematical framework would require extending our abelian group construction to higher-order derivatives. The practical implementation would need to balance the improved prediction accuracy against the increased complexity of computing and transmitting derivative information.

6.6 Control Theory Applications for Correction Stability

Our experiments revealed that algebraic reconciliation can exhibit overcorrection artifacts when temporal stability assumptions are violated, particularly visible in the characteristic double-peak error pattern during collision recovery. Future work should explore applying control theory principles, specifically PID (Proportional-Integral-Derivative) controllers, to the correction process to reduce oscillation and improve convergence characteristics.

Rather than directly applying the computed correction ε to the predicted state, a PID controller would modulate the correction based on the history of prediction errors. The proportional term would apply immediate corrections scaled by the current error magnitude. The integral term would accumulate historical errors to eliminate steady-state bias. The derivative term would dampen rapid changes to prevent overcorrection.

The controller formulation would be:

$$s_{t+1} = s_t + K_p \varepsilon_t + K_i \sum_{i=0}^{t} \varepsilon_i + K_d (\varepsilon_t - \varepsilon_{t-1})$$

where K_p, K_i, and K_d are tunable gain parameters that could be optimized per game or even adapted dynamically based on network conditions and recent prediction accuracy.

This approach could significantly improve the subjective quality of reconciliation, transforming abrupt corrections into smooth transitions that are less perceptually jarring. The control framework would also provide formal tools for analyzing stability and convergence properties, potentially enabling automated tuning of reconciliation parameters based on desired response characteristics.

6.7 Predictive Network Modeling

Current implementations react to network conditions but do not anticipate them. Future research should explore predictive models of network behavior that could inform reconciliation strategies. Machine learning techniques could identify patterns in latency variation, packet loss, and player behavior to optimize reconciliation parameters dynamically.

A neural network trained on historical network telemetry could predict upcoming latency spikes, allowing the reconciliation system to preemptively adjust correction rates or switch reconciliation strategies. During periods of predicted network instability, the system might temporarily reduce prediction aggressiveness or increase correction damping to maintain visual smoothness despite degraded synchronization.

6.8 Formal Verification and Property-Based Testing

The mathematical foundation of algebraic reconciliation enables formal verification techniques currently uncommon in game networking. Future work should develop property-based testing frameworks that automatically verify reconciliation correctness by checking algebraic properties rather than specific test cases.

Such frameworks would generate random game states, inputs, and network conditions, then verify that reconciliation maintains convergence properties regardless of the specific scenario. Properties to verify would include: eventual consistency (all clients converge given sufficient time), monotonic error reduction (corrections always reduce divergence), and bounded divergence (prediction errors remain within acceptable limits).

6.9 Conclusion

The future directions outlined in this chapter represent a roadmap for transforming algebraic server reconciliation from a promising research prototype into a practical tool for game developers. The immediate priorities, hybrid architectures and industrial validation, would establish the technique's practical viability. The optimization strategies, Merkle trees and higher-order derivatives, would extend its applicability to broader game genres. The theoretical extensions, control theory and formal verification, would provide the mathematical rigor necessary for widespread adoption.

Each direction offers unique benefits while addressing current limitations, collectively pointing toward a future where game developers can select from a rich palette of mathematically grounded synchronization techniques, each optimized for specific gameplay requirements. The algebraic approach introduced in this thesis represents not an endpoint but rather an opening into a broader exploration of how mathematical structures can inform and improve the design of networked interactive systems.

Addition	$+ := \text{State} \times \text{State} \to \text{State}$
Negative	$- := \text{State} \to \text{State}$
Zero	$0 := \text{State}$

Associative	$A + (B + C) = (A + B) = C$
Commutative	$A + B = B + C$

Error	$\varepsilon := S^t_{\text{server}} - S^t_{\text{client}}$	$t =$ on recieve, old server time
Correction	$S_{\text{corrected}} := S^t_{\text{client}} + \varepsilon$	$t =$ on receive, current time

Player	P	$\supseteq \mathbb{R}$
State	S	$\supseteq \mathbb{Z} \times \mathrm{id} \times P$

Addition	$A + B$	$= f_\cup(A, B) \cup f_\_(A, B) \cup f_\_(B, A)$
	$f(A, B)_\cup$	$= \{(x + x', i, p + p') \mid (x, i, p) \in A, ('x, i, p') \in B, x + x' \neq 0\}$
	$f(A, B)_\_$	$= \{(x, i, p) \in A \mid t = (x', i, p'), t \notin B\}$
Negative	$-A$	$= \{(-x, i, -p) \mid (x, i, p) \in A\}$

Old client state	S_{old}	$= \{(1, \text{p1}, 3)\}$
Current client state	S_{current}	$= \{(1, \text{p1}, 6)\}$
Received server state	S_{server}	$= \{(1, \text{p1}, 4)\}$
Error	ε	$= S_{\text{server}} - S_{\text{old}} = \{(0, \text{p1}, -1)\}$
Corrected	$S_{\text{corrected}}$	$= S_{\text{current}} + \varepsilon = \{(1, \text{p1}, 5)\}$

Old client state	S_{old}	$= \{(1, \text{p1}, 3), (1, \text{p2}, 0)\}$
Current client state	S_{current}	$= \{(1, \text{p1}, 4), (1, \text{p2}, 0)\}$
Received server state	S_{server}	$= \{(1, \text{p1}, 4)\}$
Negative	$-S_{\text{old}}$	$= \{(-1, \text{p1}, -3), (-1, \text{p2}, 0)\}$
Error	ε	$= S_{\text{server}} + (-S_{\text{old}}) = \{(0, \text{p1}, 0), (-1, \text{p2}, 0)\}$
Corrected	$S_{\text{corrected}}$	$= S_{\text{current}} + \varepsilon = \{(1, \text{p1}, 4)\}$

Bibliography

[1] P. Bettner and M. Terrano, “1500 Archers on a 28.8: Network Programming in Age of Empires and Beyond,” *Game Developer Magazine*, Jun. 2001, [Online]. Available: https://www.gamedeveloper.com/programming/1500-archers-on-a-28-8-network-programming-in-age-of-empires-and-beyond

[2] L. Brummitt, “Matter.js - a 2D rigid body physics engine for the web.” GitHub, 2014.

[3] T. Cannon, “GGPO.” [Online]. Available: https://www.ggpo.net/

[4] G. Fiedler, “What every programmer needs to know about game networking,” Mar. 2010, [Online]. Available: https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/

[5] T. Ford, “'Overwatch' Gameplay Architecture and Netcode,” in *Game Developers Conference (GDC)*, Mar. 2017. [Online]. Available: https://www.gdcvault.com/play/1024001/-Overwatch-Gameplay-Architecture-and

[6] L. Fuchs, *Infinite Abelian Groups, Vol. I*, vol. 36. in Pure and Applied Mathematics, vol. 36. New York and London: Academic Press, 1970.

[7] G. Gambetta, “Fast-Paced Multiplayer (Part II): Client-Side Prediction and Server Reconciliation.” [Online]. Available: https://www.gabrielgambetta.com/client-side-prediction-server-reconciliation.html

[8] R. C. Merkle, “A Certified Digital Signature,” *Communications of the ACM*, vol. 23, no. 4, pp. 295–300, 1980, doi: 10.1145/359340.359342.

[9] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free Replicated Data Types,” *INRIA*, no. RR-7687, 2011, [Online]. Available: https://hal.inria.fr/inria-00609399v1/document

[10] M. Stallone, “8 Frames in 16ms: Rollback Netcode in Mortal Kombat and Injustice 2,” in *Game Developers Conference (GDC)*, Mar. 2016. [Online]. Available: https://www.gdcvault.com/play/1023225/8-Frames-in-16ms-Rollback

[11] Valve Developer Community, “Source Multiplayer Networking.” Accessed: Sep. 05, 2024. [Online]. Available: https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking

[12] wiki.vg, “Protocol.” Accessed: Sep. 05, 2024. [Online]. Available: https://wiki.vg/Protocol